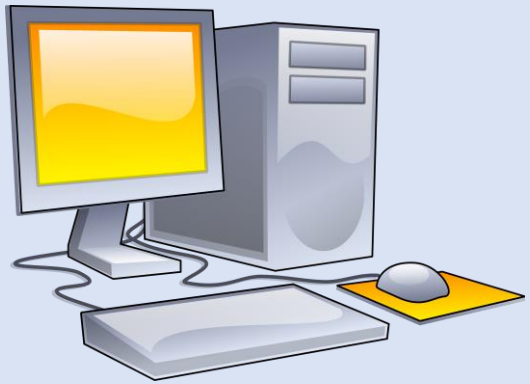
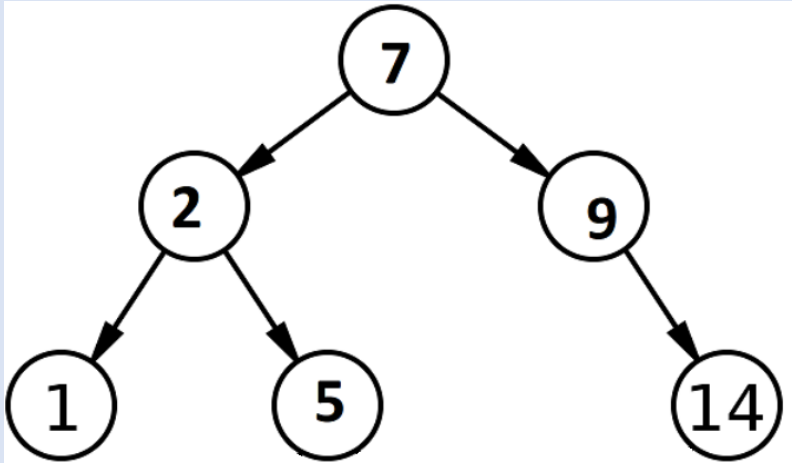
Binary Trees
&
Binary Search Trees

COP 3502
Spring 2025
Department of Computer Science
University of Central Florida
Dr. Steinberg



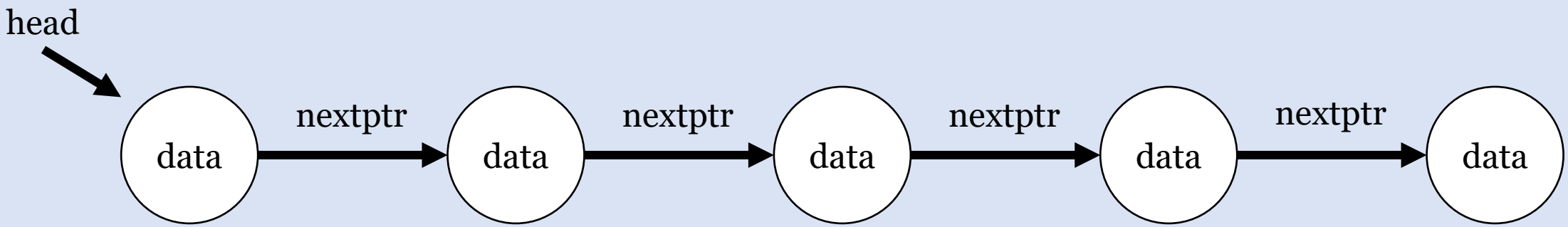


What we have learned so far in this course!

- So far, you have learned about the following elementary data structures
  - Arrays
  - Linked Lists
  - Stacks
  - Queues
- These data structures can be categorized as a **linear data structure**.
  - This is due to the linear runtime constraints of some of the operations.
    - Stack Array Implementation if Size Doesn’t Matter Can have a linear runtime (push)!
- We should be wondering if there is a data structure that wouldn’t have linear runtime constraints always.

The Linear Data Structures Implementation

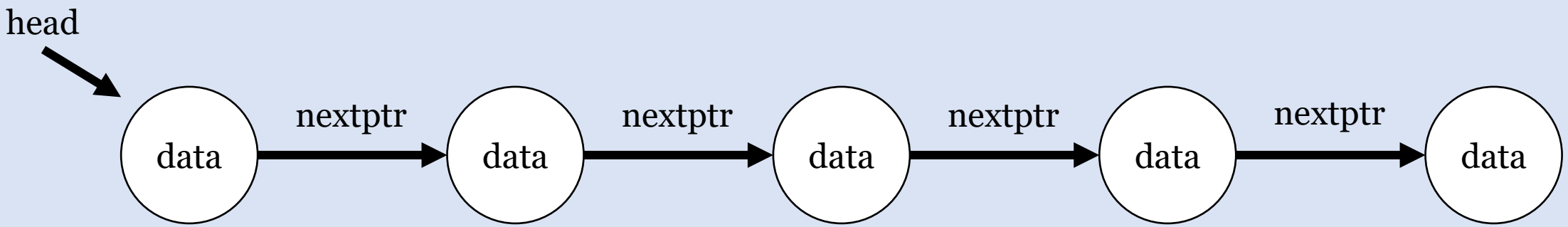
- These past few classes we have been implementing ADTs based on the linked list.
- Let’s assume we are looking at the singly linked list data structure.
- Most operations with the singly linked list requires some sort of linear runtime.



The Linear Data Structures Implementation

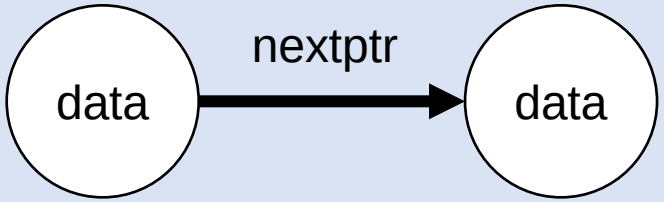
- These past few classes we have been implementing ADTs based o
- Let's as t data structur
- Most op some sort of linear ru

We can do even better!!!



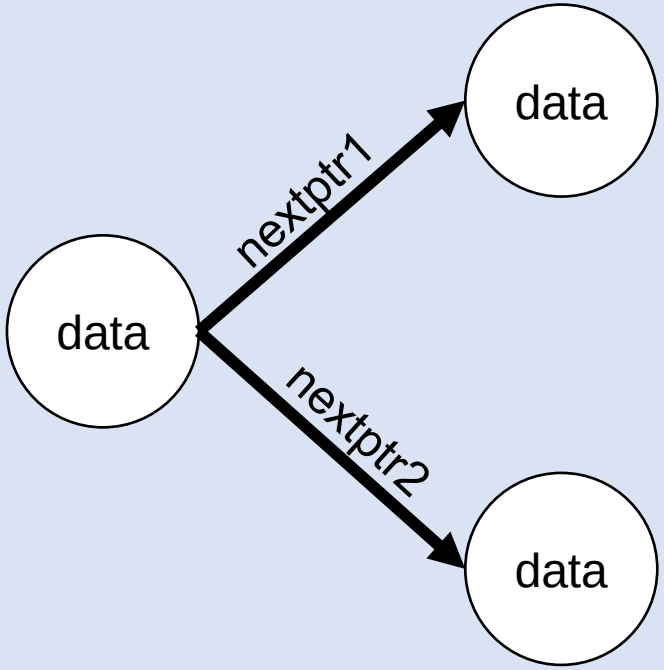
Modify the Node Traversal

A Node with Only One Pointer Option



Only 1 Option To Traverse

A Node with Only Two Pointer Options



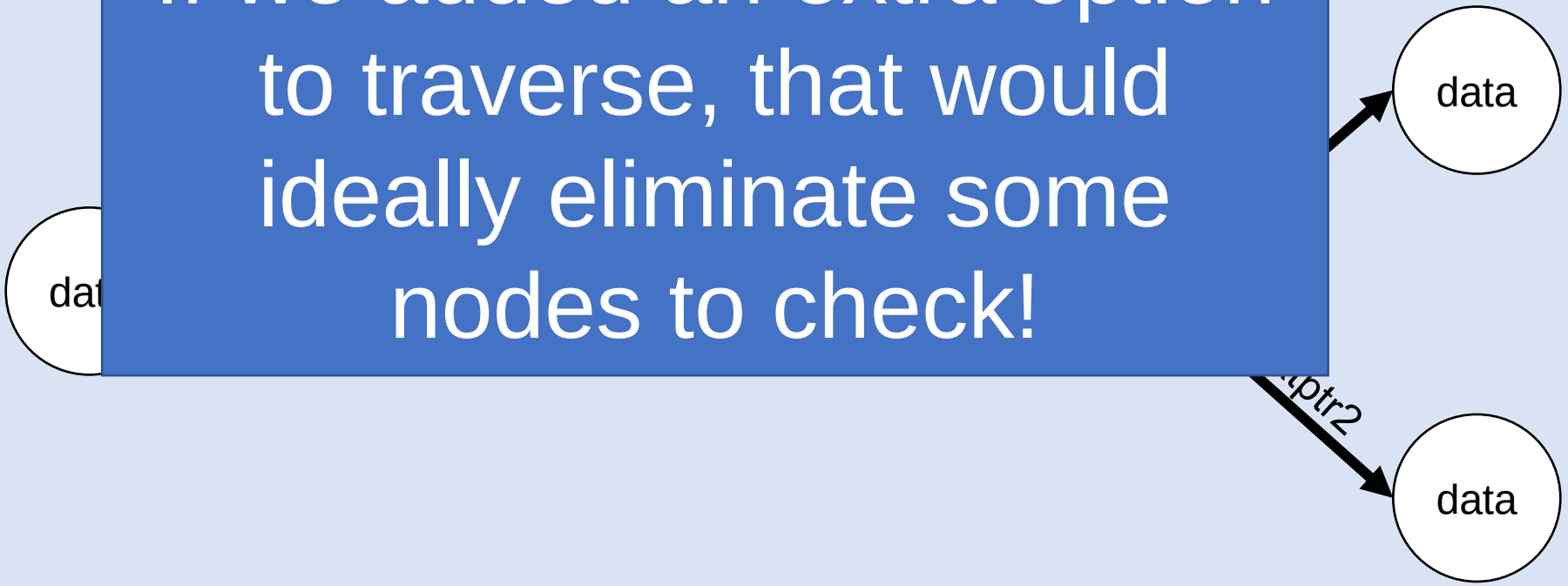
Only 2 Options To Traverse

Modify the Node Traversal

A Node with Only One Pointer Option

A Node with Only Two Pointer

If we added an extra option to traverse, that would ideally eliminate some nodes to check!

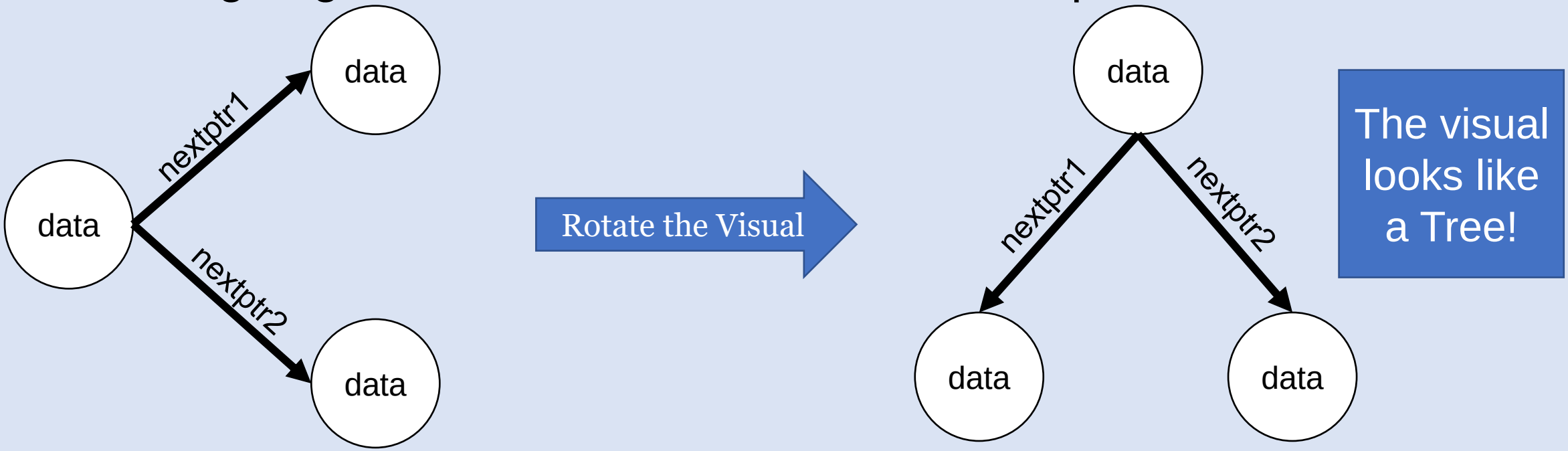


Only 1 Option To Traverse

Only 2 Options To Traverse

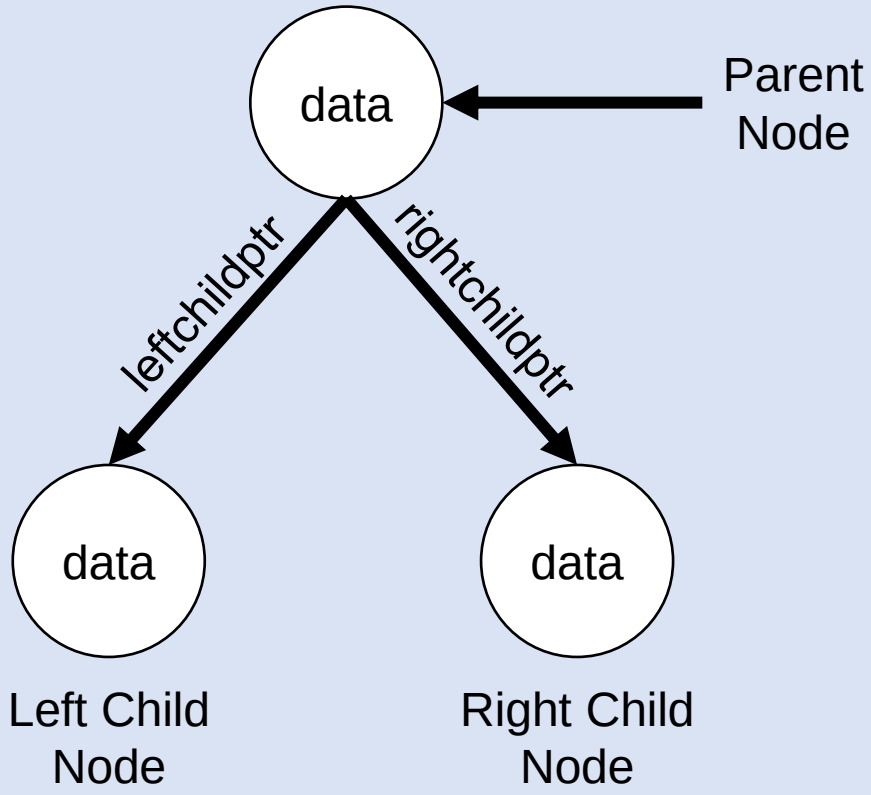
The Node with Two Pointers

- We just saw that we can reconstruct a node to have potentially multiple pointers.
- This can lead to less traversal operations of the data structures.
- We are going to observe a node with two next pointers.



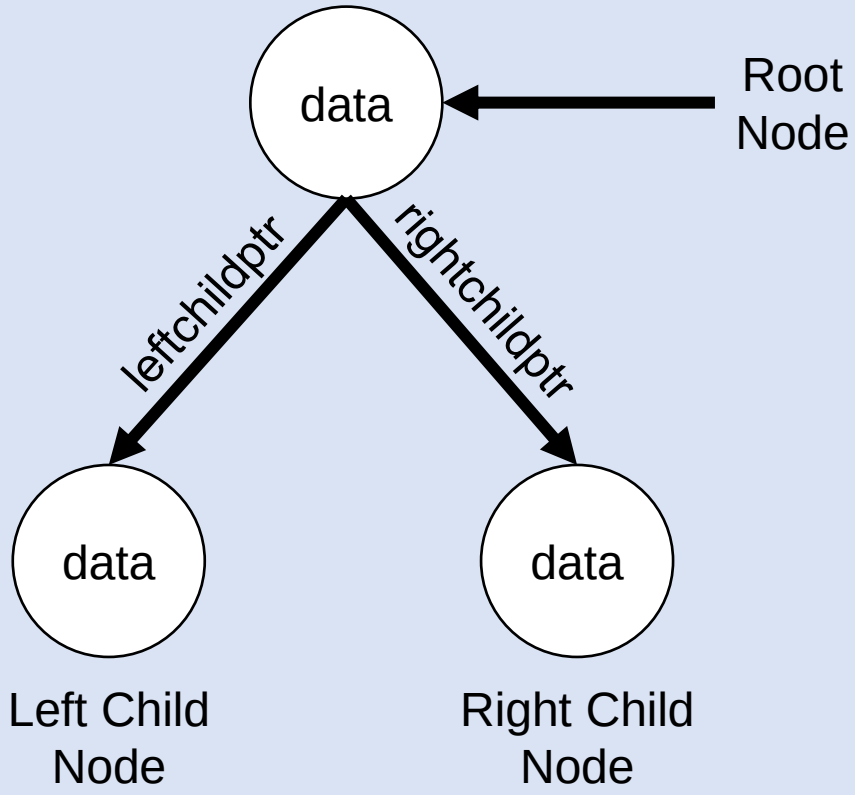
The Tree Structure Visual

- When discussing the Tree structure, we will use a lot of family related terminology to describing the structure. Think of a Family Tree!
  - Parent
  - Children
- A tree structure allows a node to have multiple pointers.
- This makes the Tree a nonlinear data structure.
- In this course, we will only observe the **Binary Tree** Structure.
  - That means a node will have at **<u>MOST</u>** 2 children pointers!



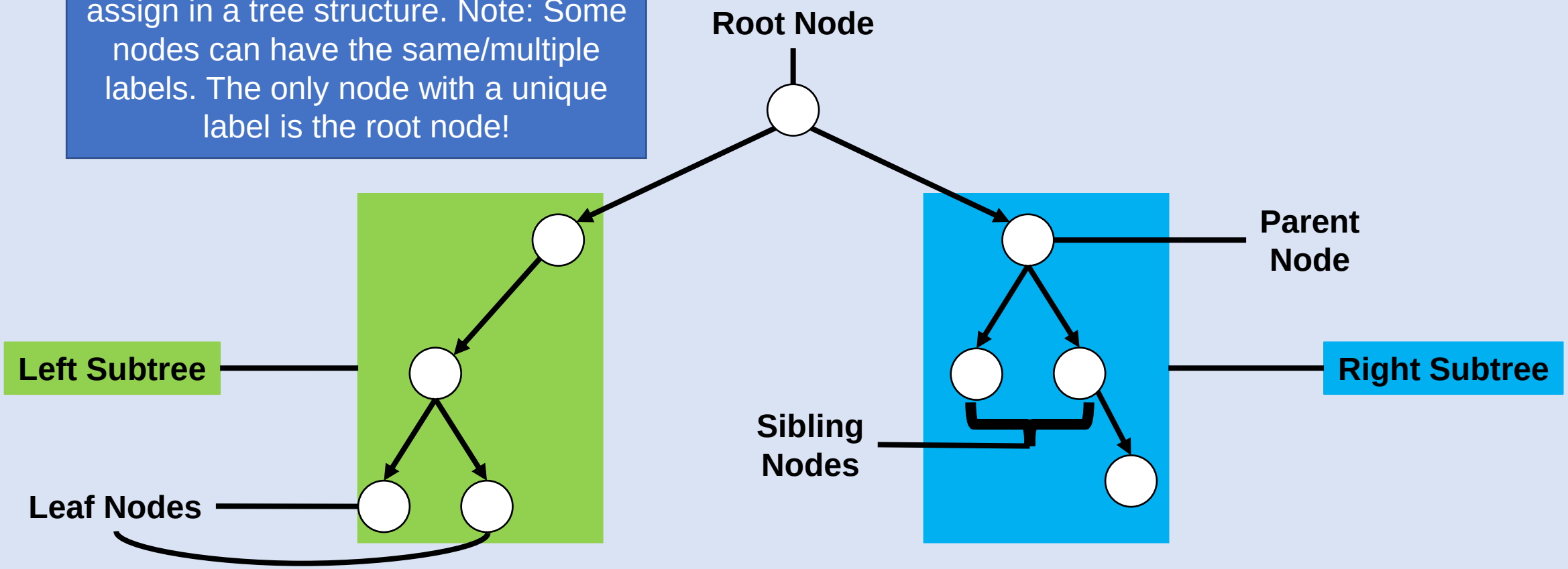
Tree Structure Terminology

- Root – The first node of entry into the Tree Structure (similar to the head of a linked list).
- The left child is the first node in the left subtree.
- The right child is the first node in the right subtree.
- The two pointer values stores the address of the **children** nodes.
  - The children of a respective node are called siblings!
  - We also call the pointers edges.
    - Left Edge
    - Right Edge
- A node with NO children is called a **leaf**.
- Subtree – A non-root node that can represent a smaller subset of the tree.



Tree Structure Visual with Terminology

This sample shows all the labels we assign in a tree structure. Note: Some nodes can have the same/multiple labels. The only node with a unique label is the root node!



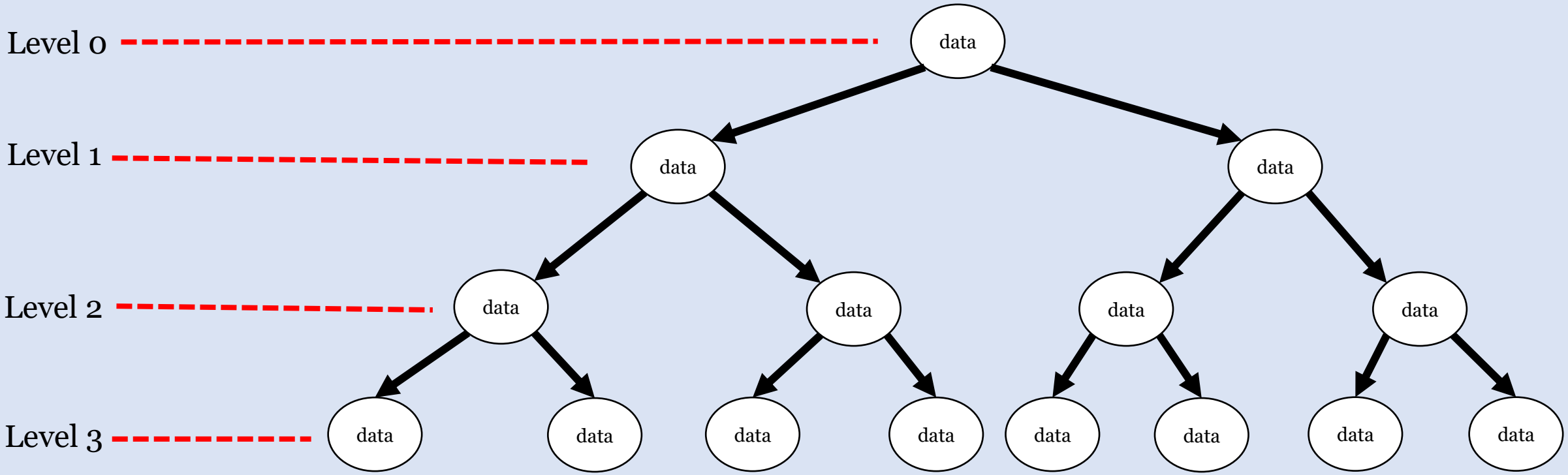
More Tree Structure Terminology

- Internal nodes – nodes of a tree that have at least one child
  - Internal nodes are not leaves!
- Degree – the total number of children a node points to.
  - We will observe the following values for Binary Trees
    - 0 children
    - 1 child
    - 2 children
- Level – since we are observing the tree from a vertical perspective, we need to make sure we classify a group of nodes based on their current vertical height perspective
  - Think of tree as a hotel building. Hotels have many “floor levels”.
- Height – the number of edges (generally the longest path) from a leaf node to any other node in the tree.
- Depth – the number edges from the root node to a particular node
  - Depth of Tree is considered the longest path from a root node to a leaf node.

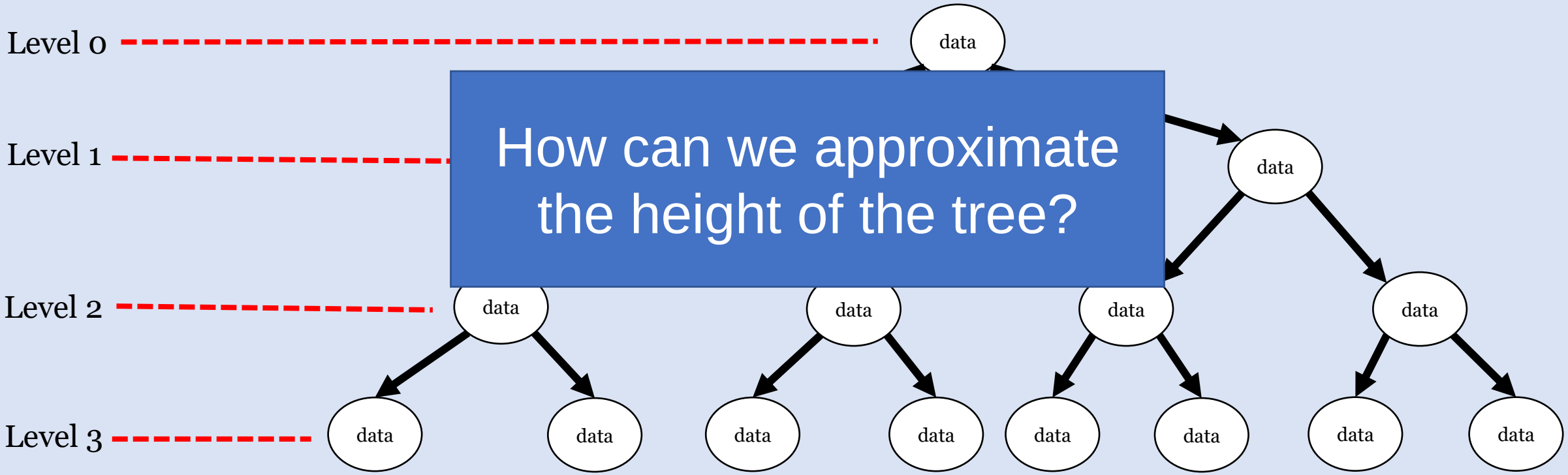
Traversing the Tree

- When we traversed a linked list, we would count the number of nodes.
- How do we properly count the number of nodes when a traversing a tree (since there will be nodes skipped)?
- Generally, we are interested in the longest path from the root node to a leaf node.
- We can use the height the tree!

The Height of a Tree

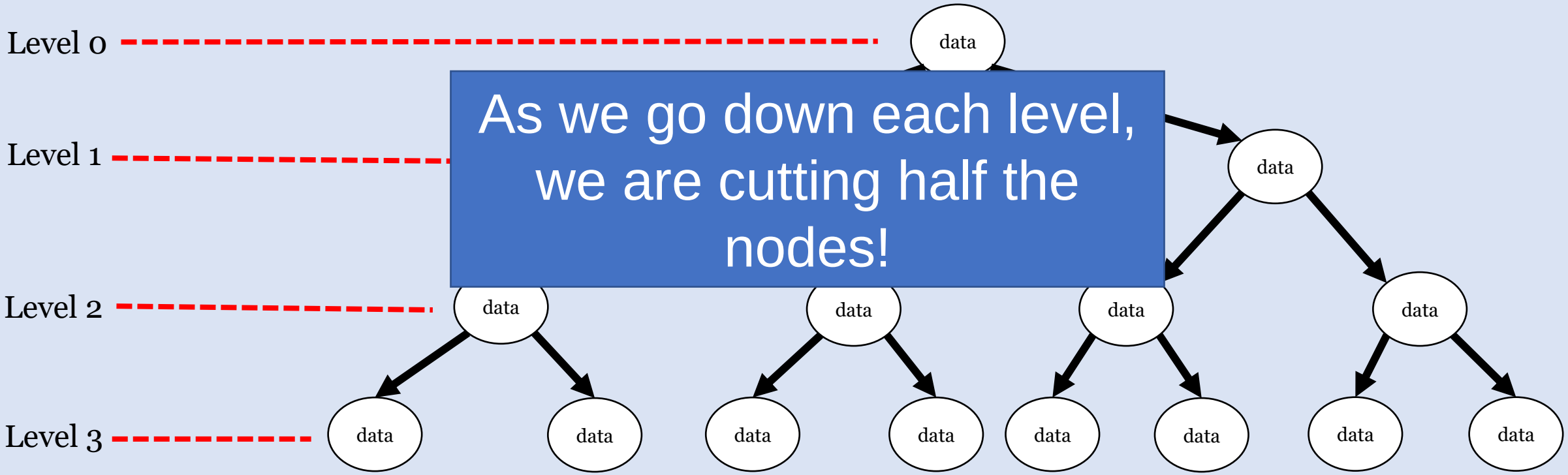


The Height of a Tree



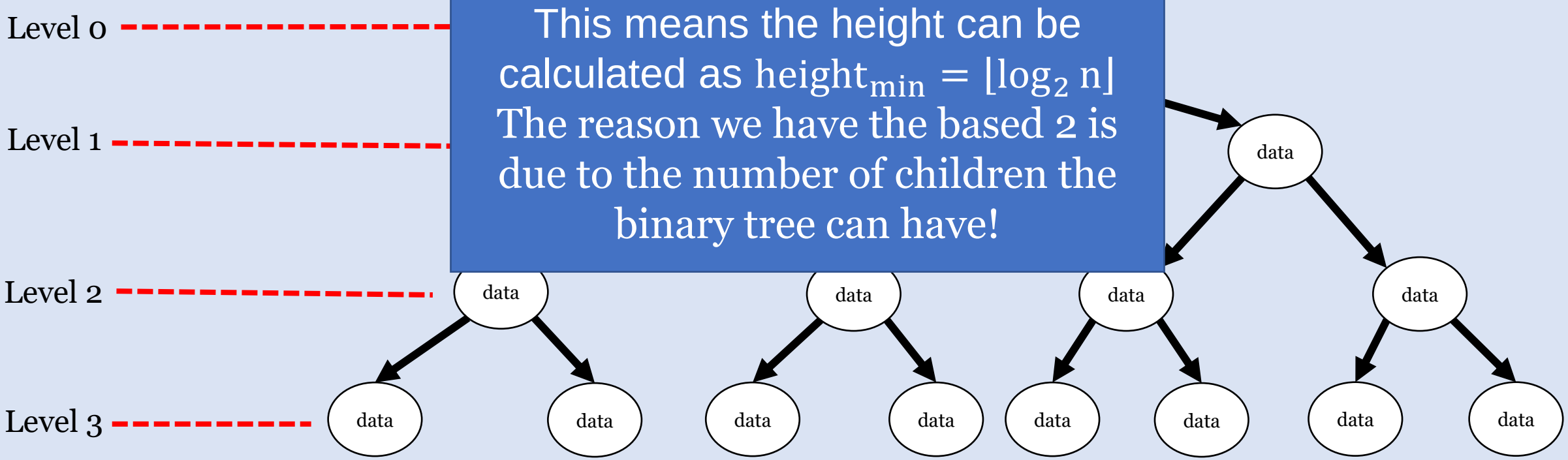
How can we approximate the height of the tree?

The Height of a Tree



The Height of a Tree

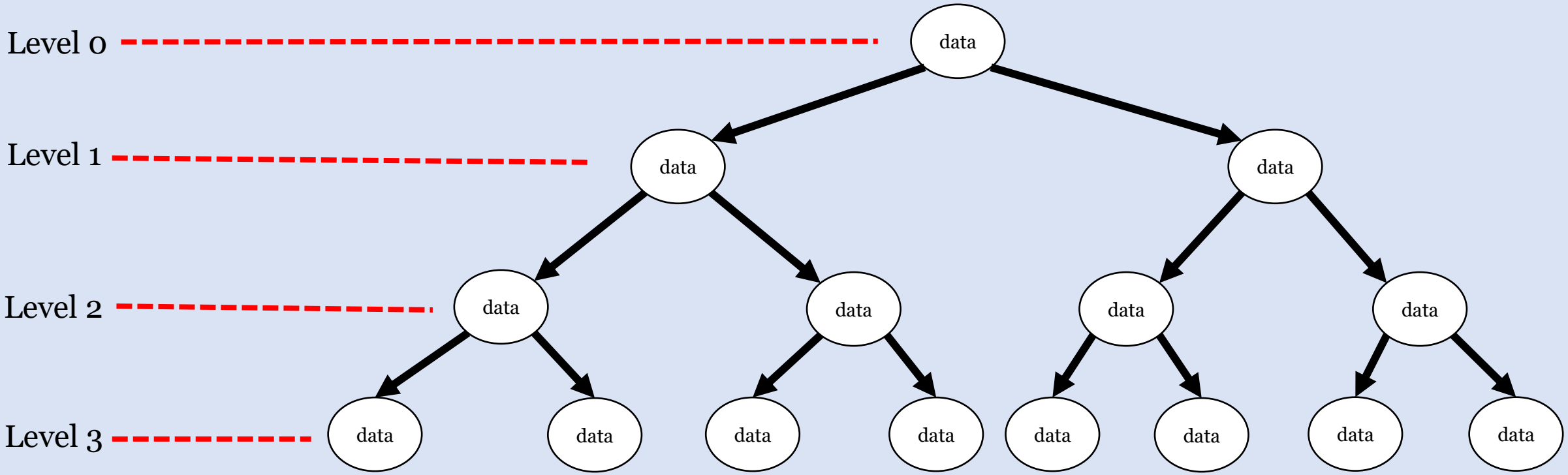
This means the height can be calculated as $\text{height}_{\text{min}} = \lfloor \log_2 n \rfloor$
The reason we have the based 2 is due to the number of children the binary tree can have!



The Height of a Tree

Here there are 15 nodes.

$$\text{height}_{\text{min}} = \lfloor \log_2 n \rfloor$$
$$\text{height}_{\text{min}} = \lfloor \log_2 15 \rfloor$$
$$\text{height}_{\text{min}} = \lfloor 3.907 \rfloor$$
$$\text{height}_{\text{min}} \approx 3 \text{ levels}$$



Implementing a Tree in C

- Now that we have discussed various terms with the tree data structure, we need to understand how to properly translate the node into C code!
- From the previous data structures, we have always two versions (Array and Linked List).
- We are going to observe the Linked List implementation only!
- We will come back to the Array implementation when discussing the Heap. (Stay Tuned!)

Linked List Implementation of a Tree

- In this course, we are only going to observe a node having at most two children!
  - Binary Trees
- We can create a similar struct definition as the linked list with one slight addition and modifications.
  - Nextptr is not the proper term to use.
    - leftchild and rightchild
  - Adding the extra pointer!

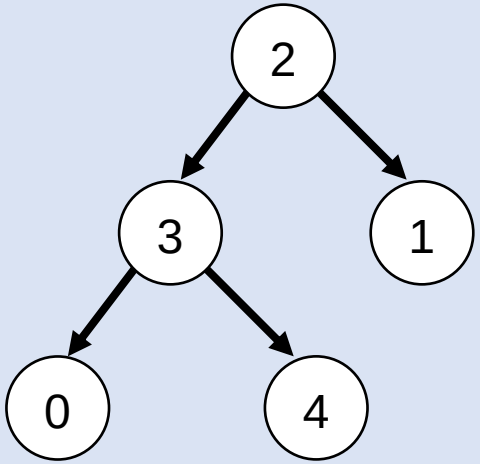
```
typedef struct node_s{

    int data;
    struct node_s * leftchild;
    struct node_s * rightchild;

}treenode_t;
```

Binary Trees

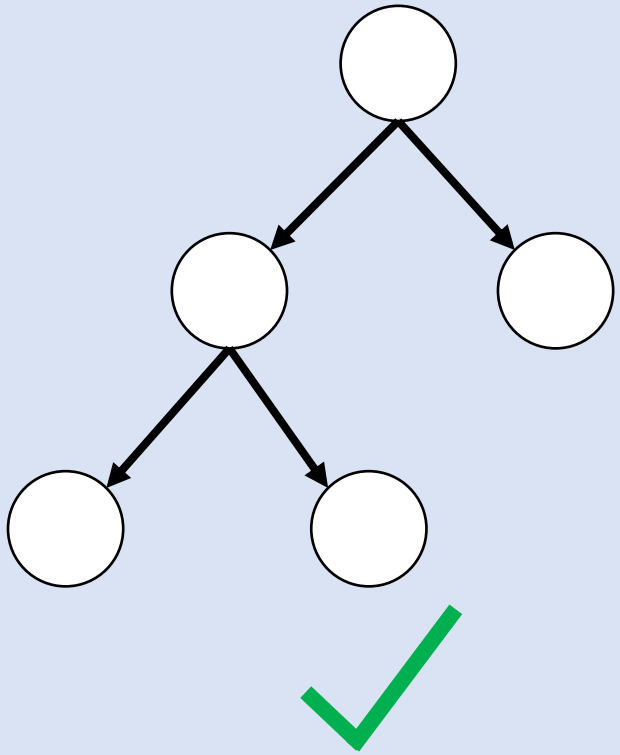
- Now that we have covered a Tree Structure and terminology, we are going to cover two types of trees.
- Binary Trees are tree data structures where each node can have most two children.
- Order of data value is not considered!



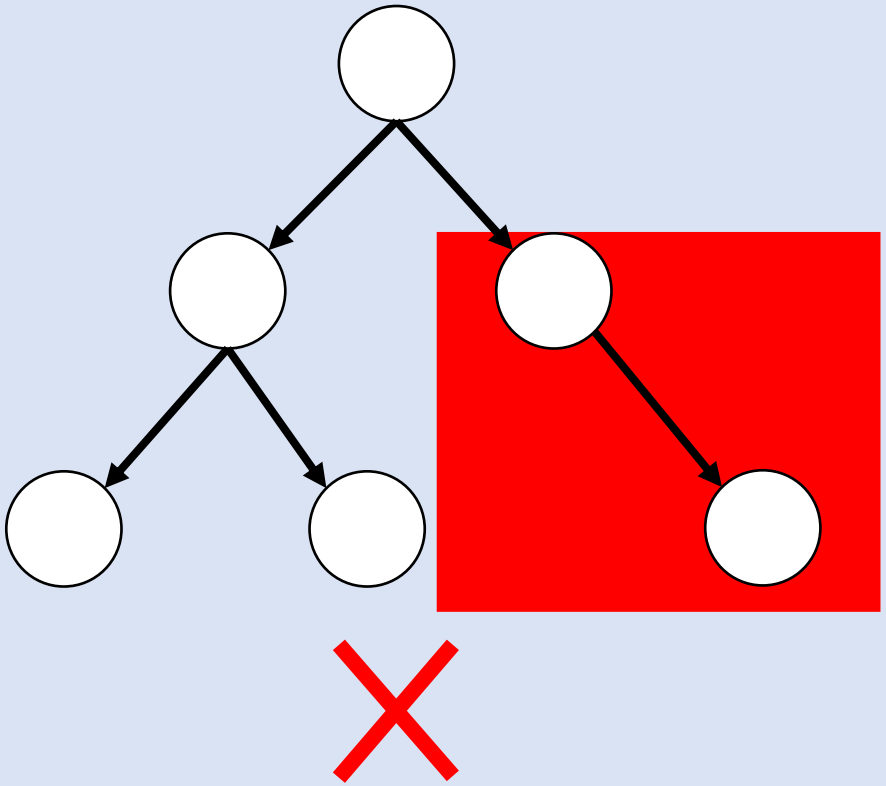
Types of Trees

- Full Binary Trees
  - Every node has either two OR no children
- Complete Binary Trees
  - Every Level Must be Filled
  - All leaves must be on the left filled
  - The last leaf must not have a right sibling

Full Binary Tree Examples

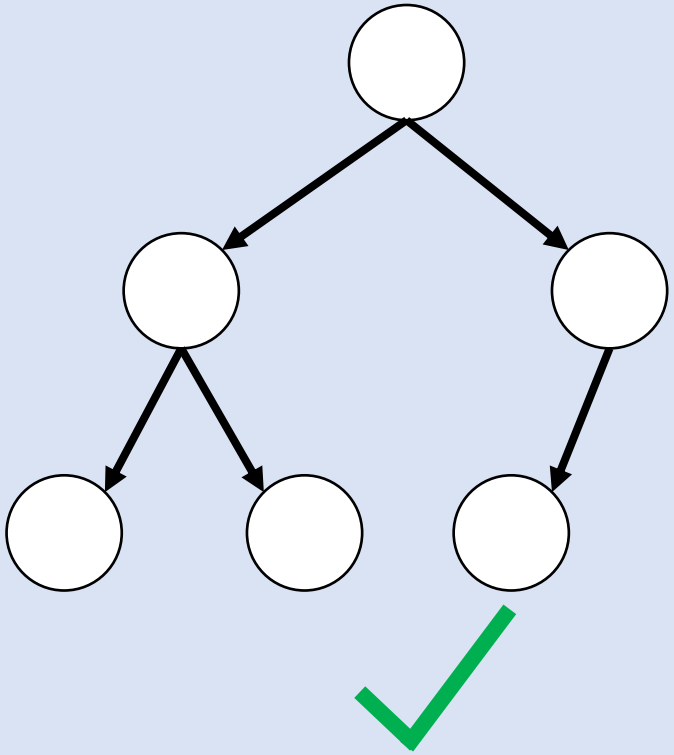


Each node has either the 2 children or no children.

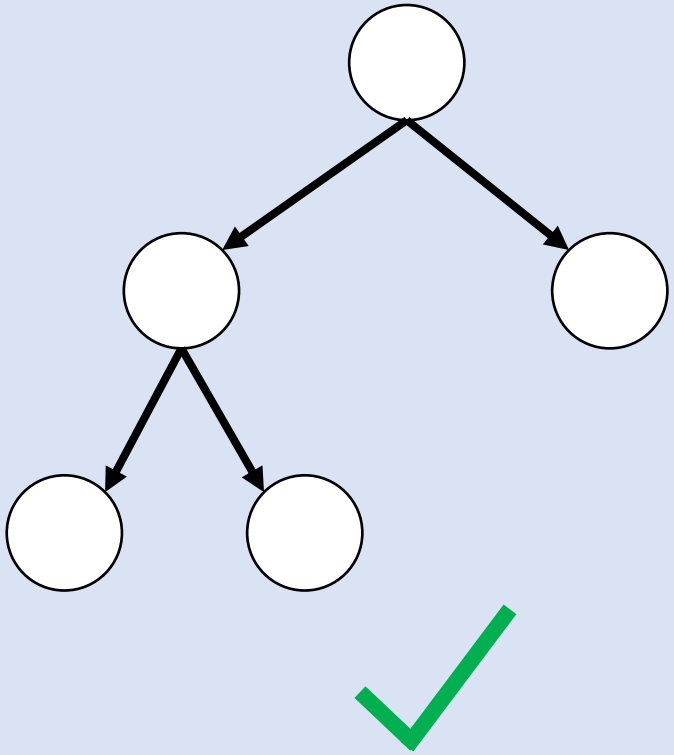


The shaded region shows a node with only 1 child. This is not a full binary tree.

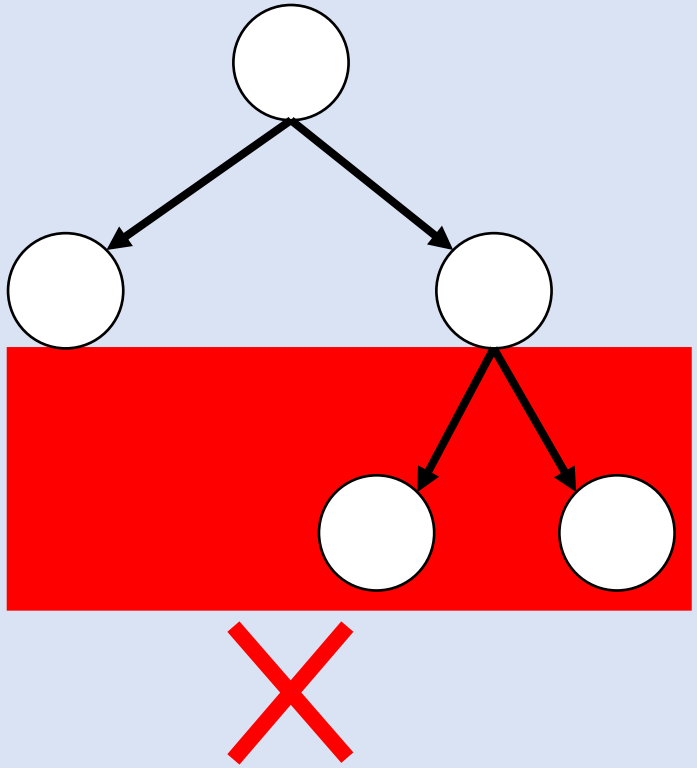
Complete Binary Tree Examples



Every level filled, and the leaves are left filled.



Every level filled, and the leaves are left filled.



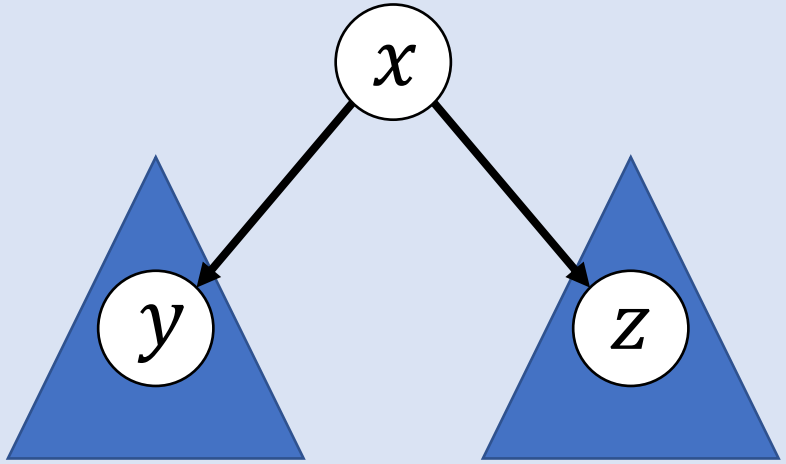
Leaves are not left filled!

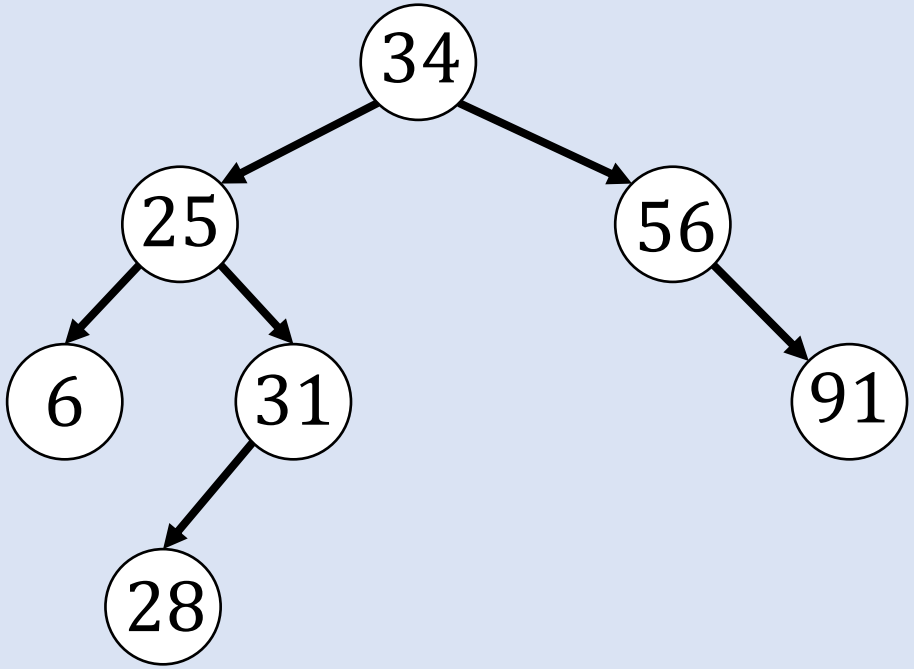
Binary Search Tree

- The Binary Trees we observed don’t have many rules of which nodes are stored.
- If we were to use a simple Binary Tree, how can we properly know which pointer to traverse.
- In this course we will primarily focus on the Binary Search Tree.
- The Binary Search Tree (BST) is a special Binary Tree that requires the characteristics of how data is stored.

Binary Search Tree Property

- The BST property states that for any node x that belongs into the BST.
  - For any node y in $x's$ left subtree, $y.data \leq x.data$
  - For any node z in $x's$ right subtree, $x.data \leq z.data$





Common Operations of the Binary Search Tree

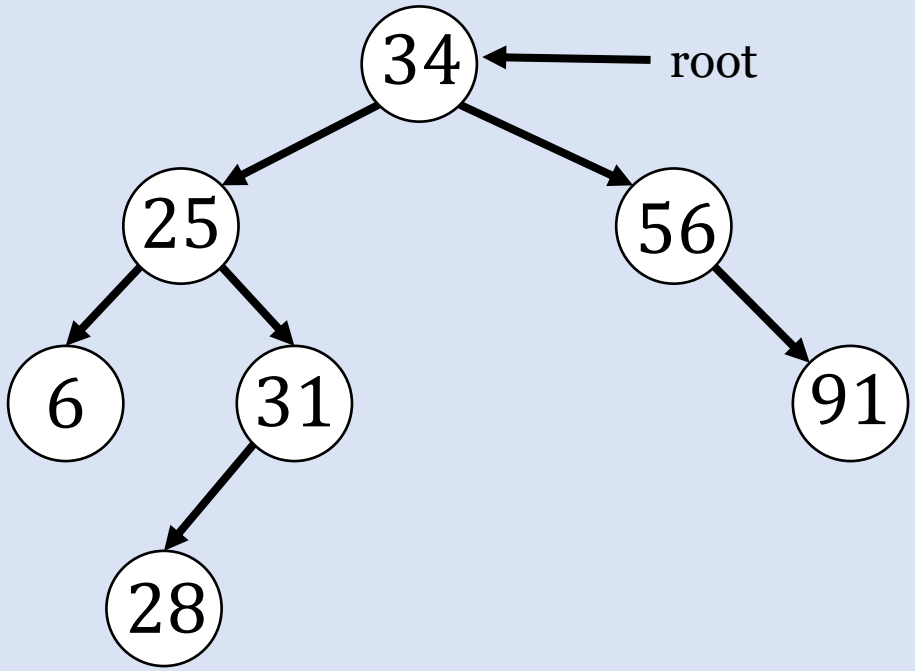
- The BST has a couple of operations that will allows to maintain the BST property.
  1. Insertion
  2. Deletion
  3. Search
  4. Traversal
     - inorder
     - preorder
     - postorder

Insertion

- Every data structure we have learned always has some insertion operation. The binary search tree has an insertion operation.
- Insert a new node into the BST structure.
- The big question: How do we properly insert a new node (with a new value) that will allow the BST to maintain the property?
- The idea is quite simple!
  - We start at the root!
  - Traverse the tree until we reach a null value.
    - Check to see if value to be inserted is greater than value in tree.
      - If so, traverse to the right child
      - Else, traverse to the left child
- This can be performed Iteratively and Recursively.
- Runtime:
  - Worst-Case: O(n)
  - Average-Case: O(h)

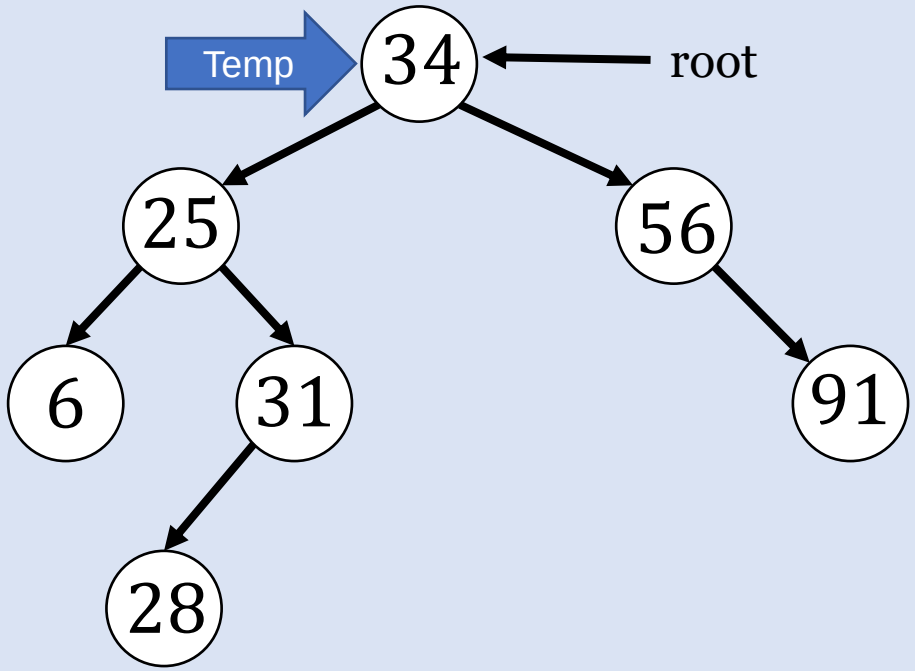
Insertion Example

Insert a new node with the value 33



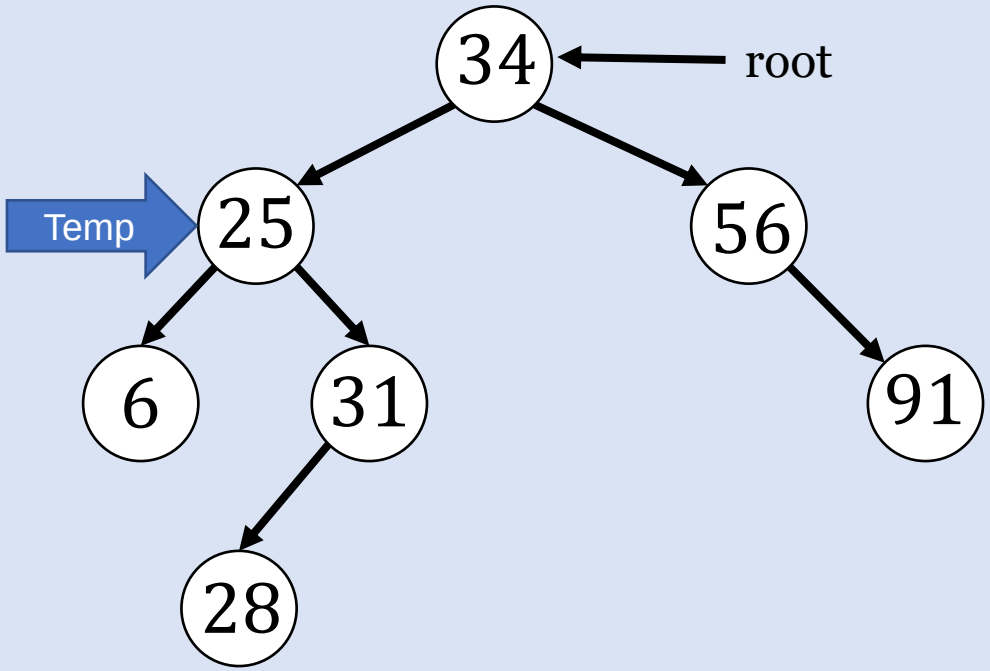
Insertion Example

Insert a new node with the value 33



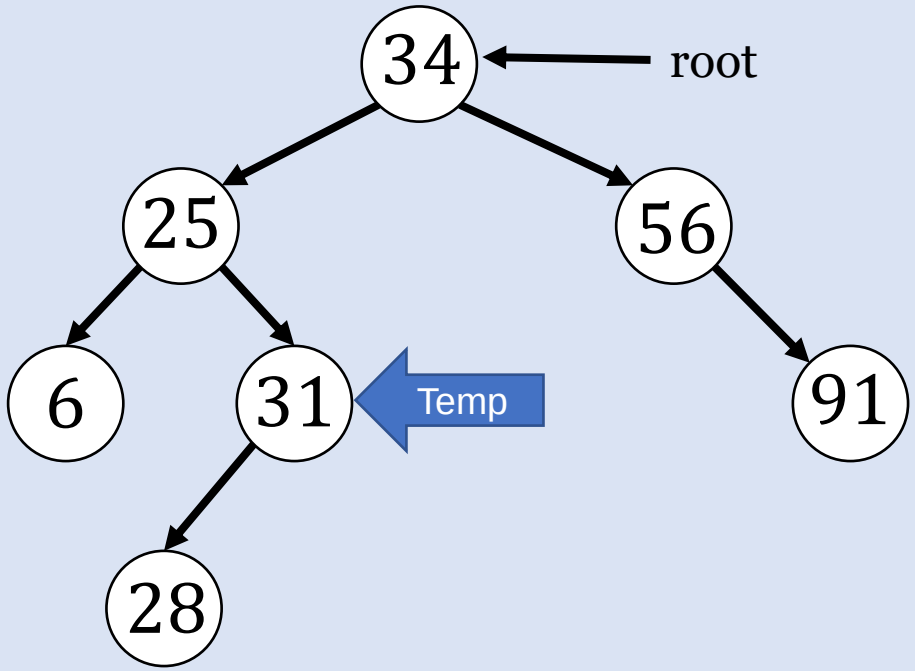
Insertion Example

Insert a new node with the value 33



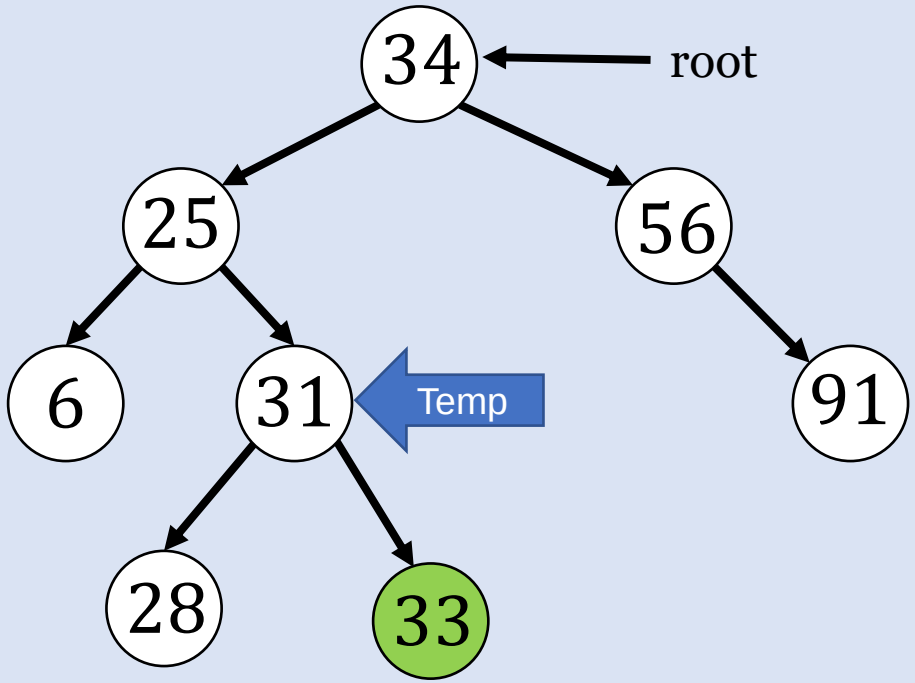
Insertion Example

Insert a new node with the value 33



Insertion Example

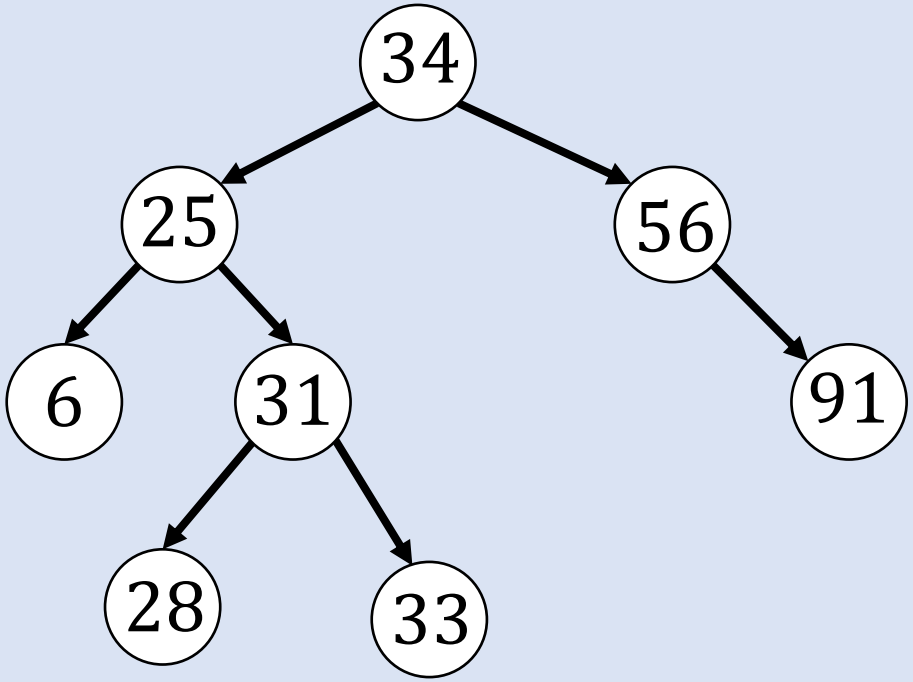
Insert a new node with the value 33



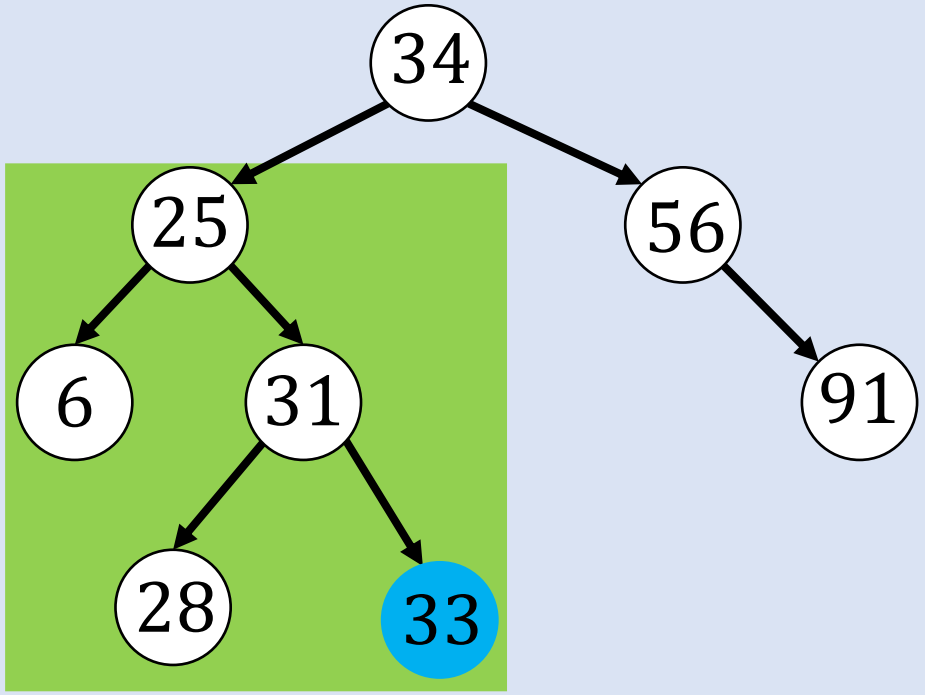
Predecessor and Successor

- The predecessor of a node is the considered the left subtree’s right most child (maximum value if the subtree).
- The successor of a node is the considered the right subtree’s left most child (minimum value if the subtree).
- These nodes are very useful when need to perform the deletion operation.

Predecessor Example

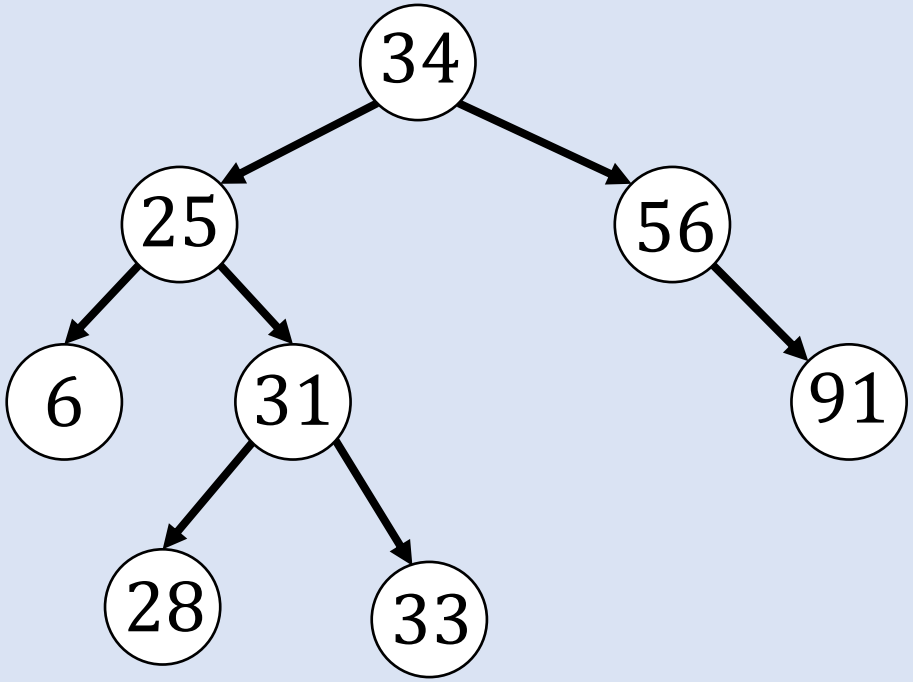


Find the predecessor of 34.

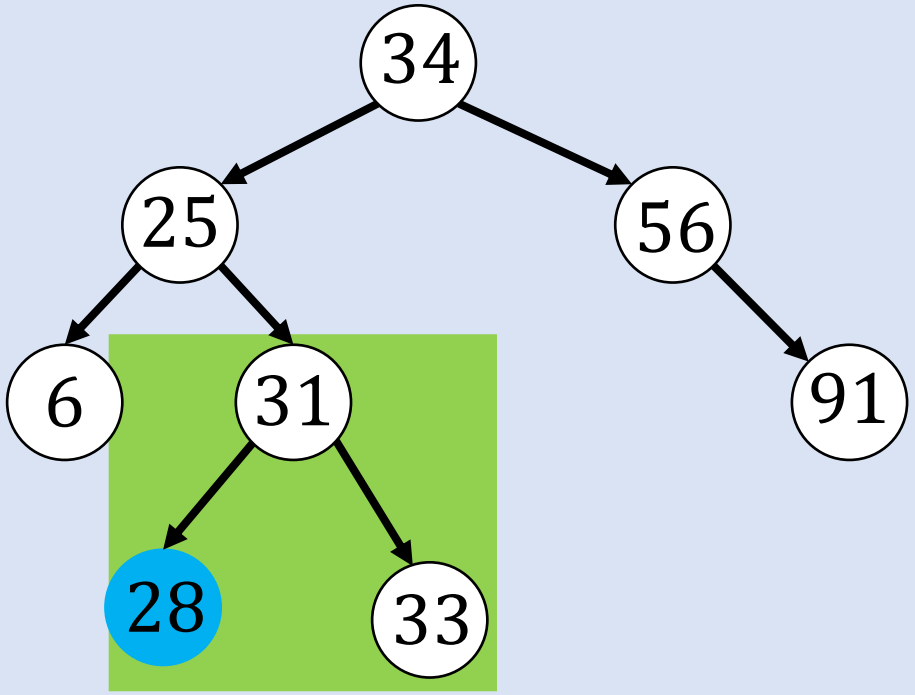


The predecessor is 33.

Successor Example



Find the predecessor of 25.



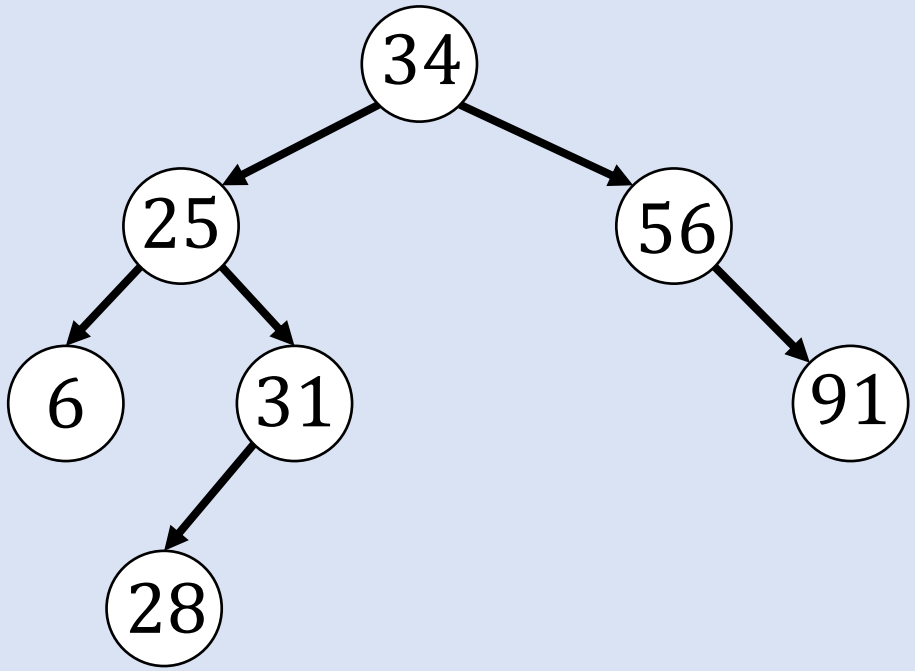
The predecessor is 28.

Deletion

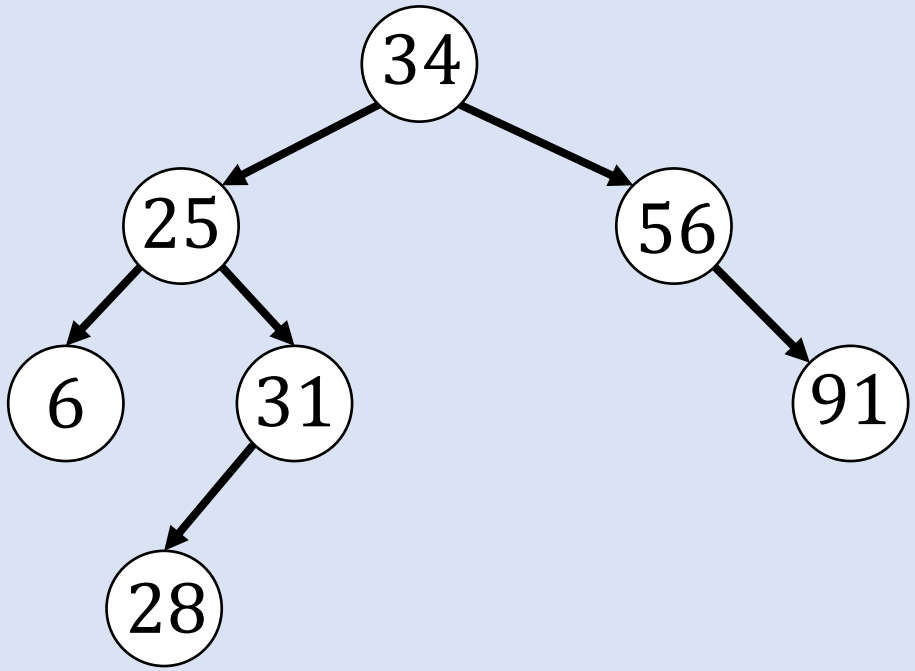
- There will be scenarios where the binary search tree needs to remove a node from the structure.
- Before we remove the node, we need to see if the value exists in the binary search tree.
- Can we just simply remove a node? NO!
  - We must ensure the BST property is maintained!
- There are 3 scenarios to consider
  - Scenario 1:
    - Node to delete is just leaf
      - Remove
  - Scenario 2:
    - Node to delete has only one child
      - Copy child node and replace node to be deleted
      - Delete the child node
  - Scenario 3:
    - Node to delete has two children
      - Find the Successor and replace it to node to be deleted
      - Delete old successor node
- Runtime:
  - Worst-Case: O(n)
  - Average-Case: O(h)

Deletion Scenario 1

Delete the node with value 91.

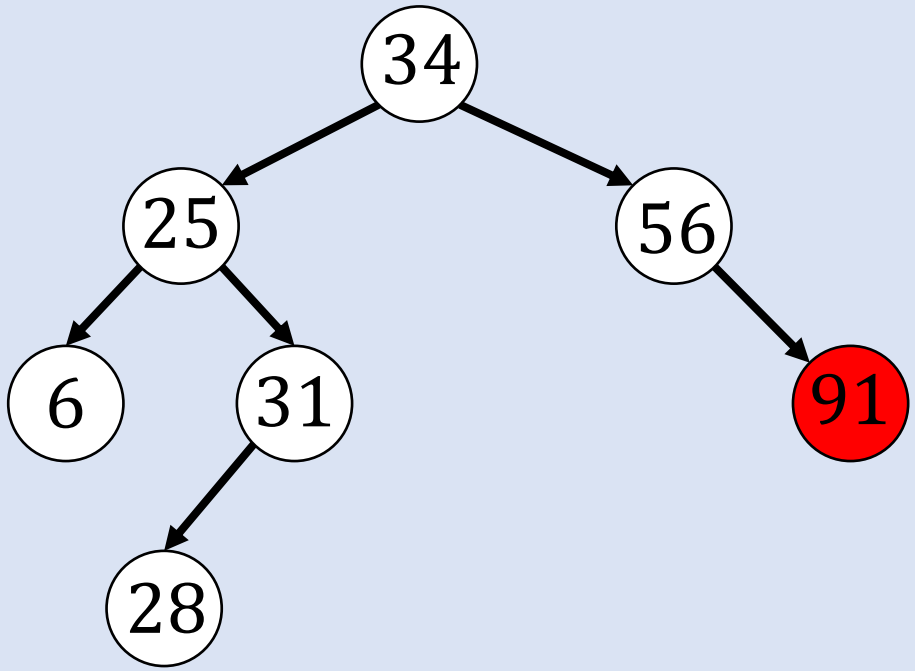


Deletion Scenario 1



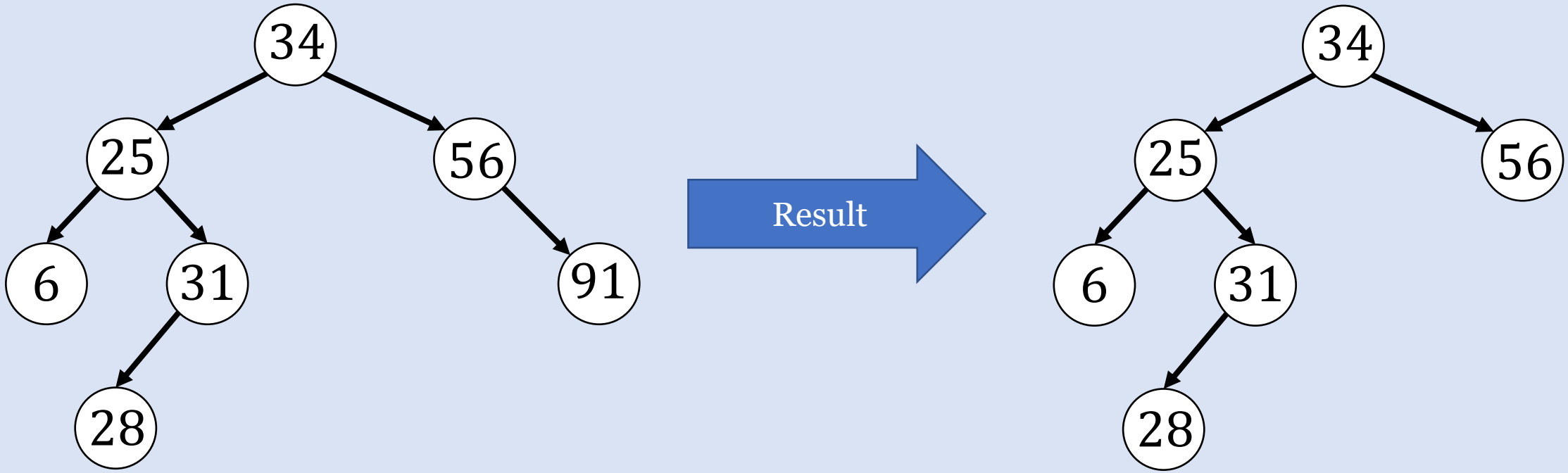
91 is a leaf

Deletion Scenario 1



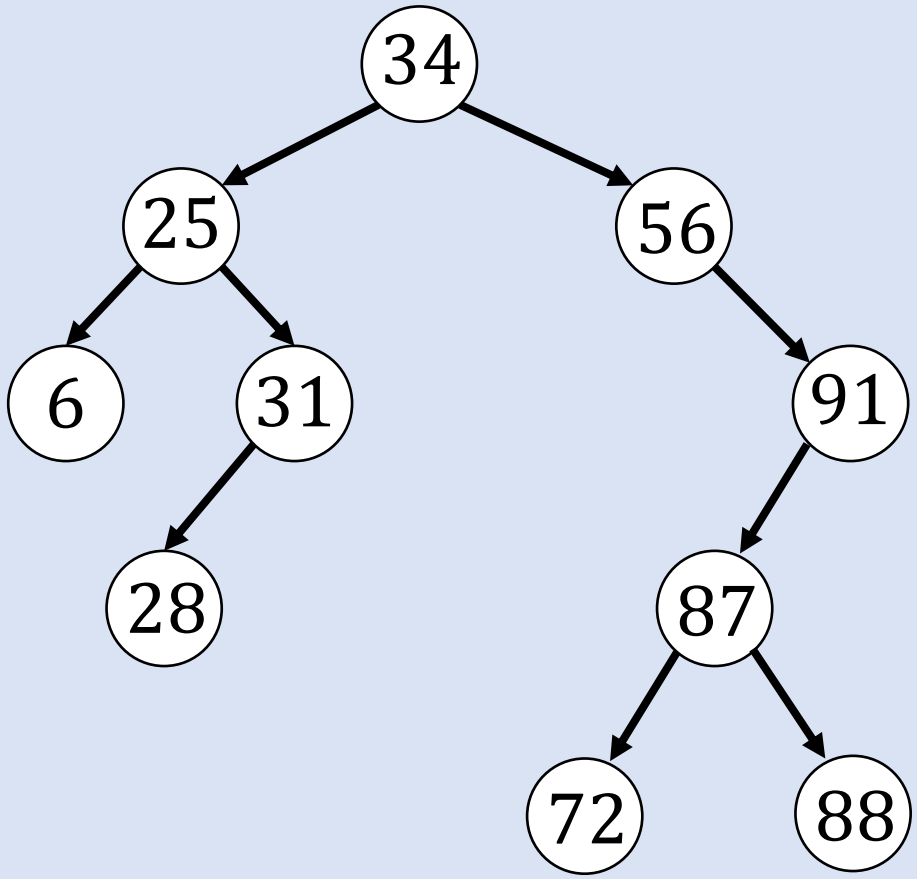
We can just simply delete the node without violating the BST property.

Deletion Scenario 1



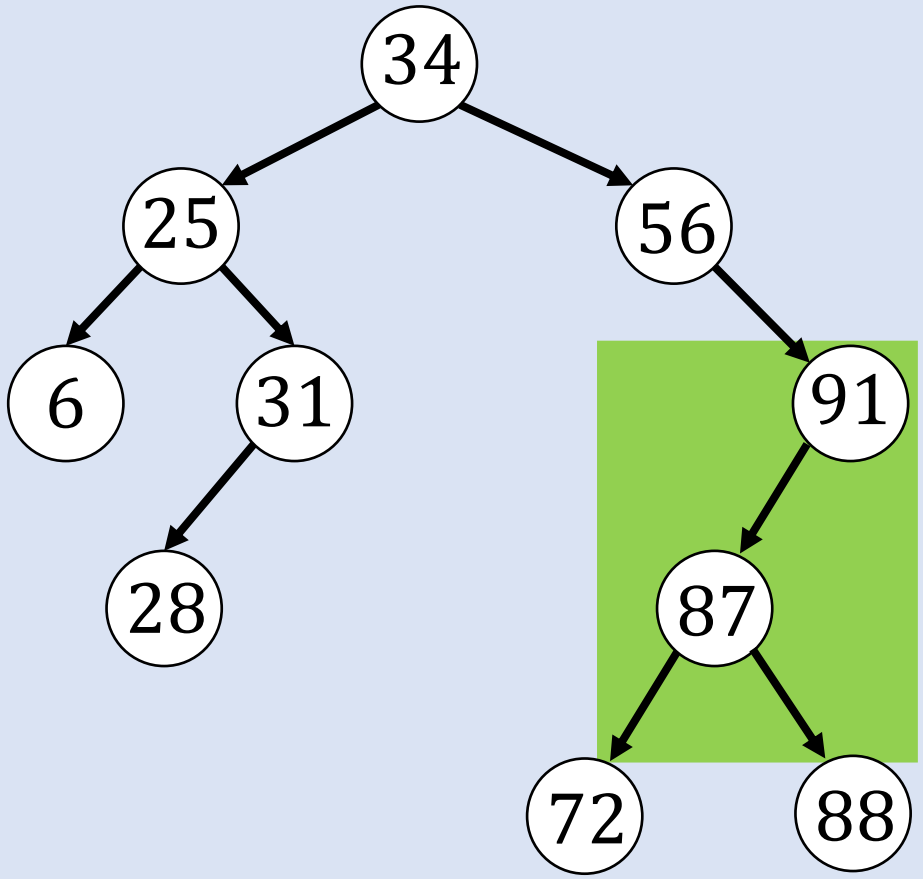
Deletion Scenario 2

Delete the node with value 91.



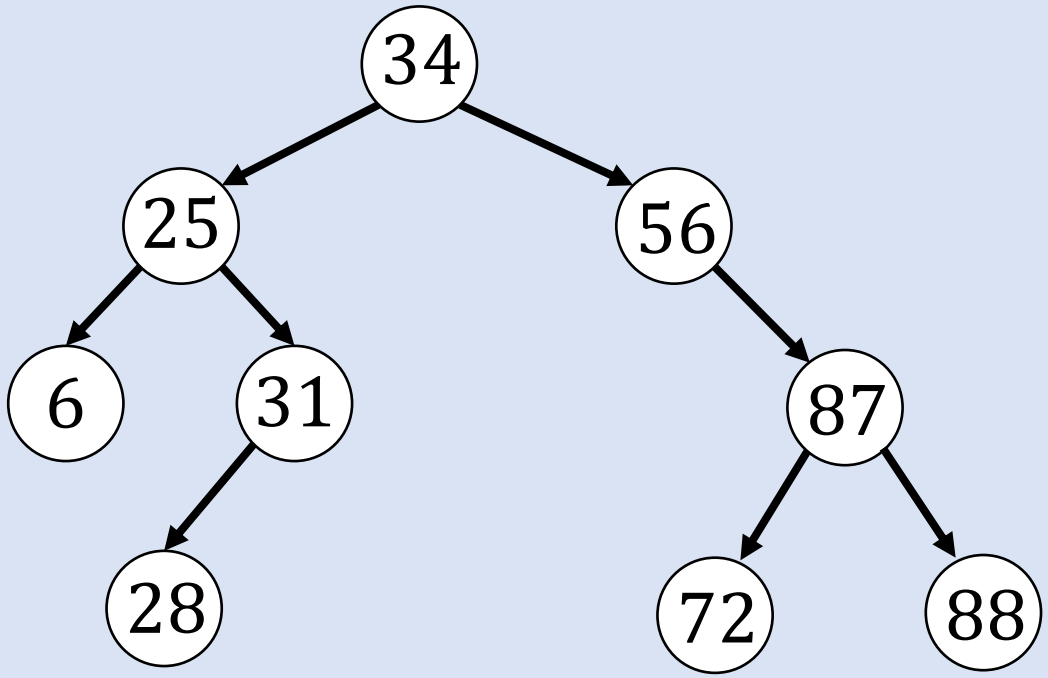
Deletion Scenario 2

91 has only 1 child

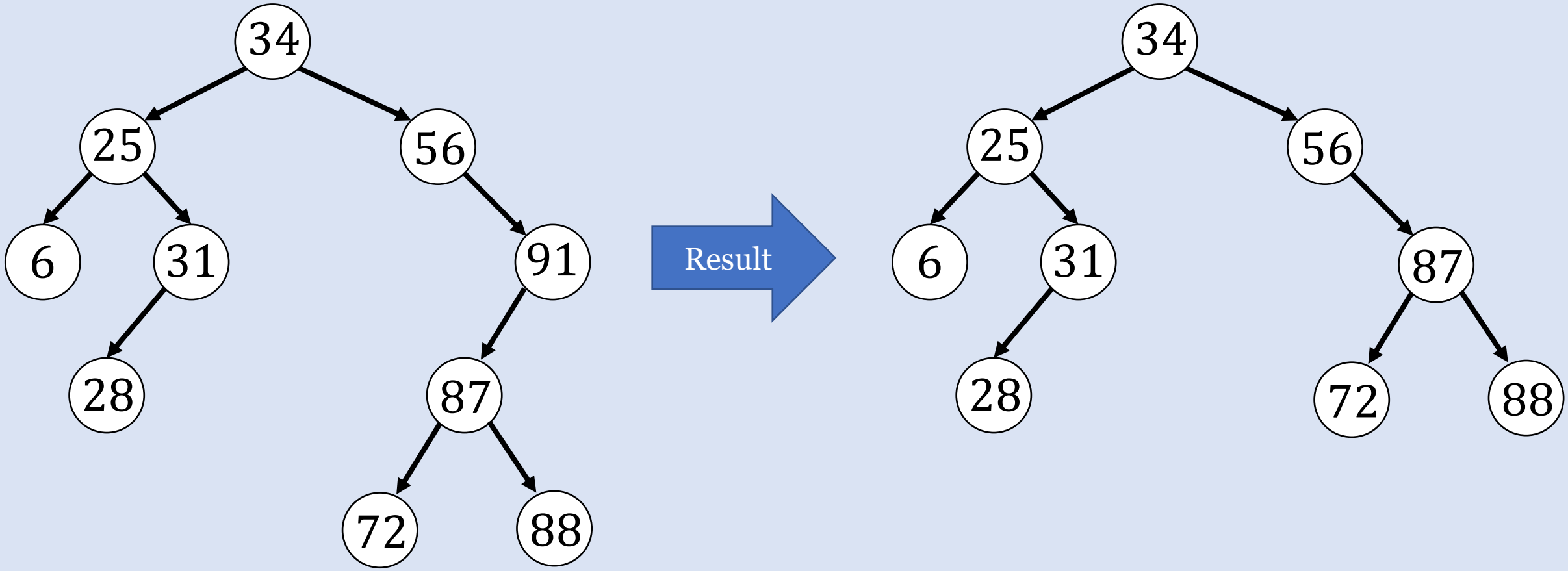


Deletion Scenario 2

We just must reassign 56 to point to 87. Simply, overwrite 91.

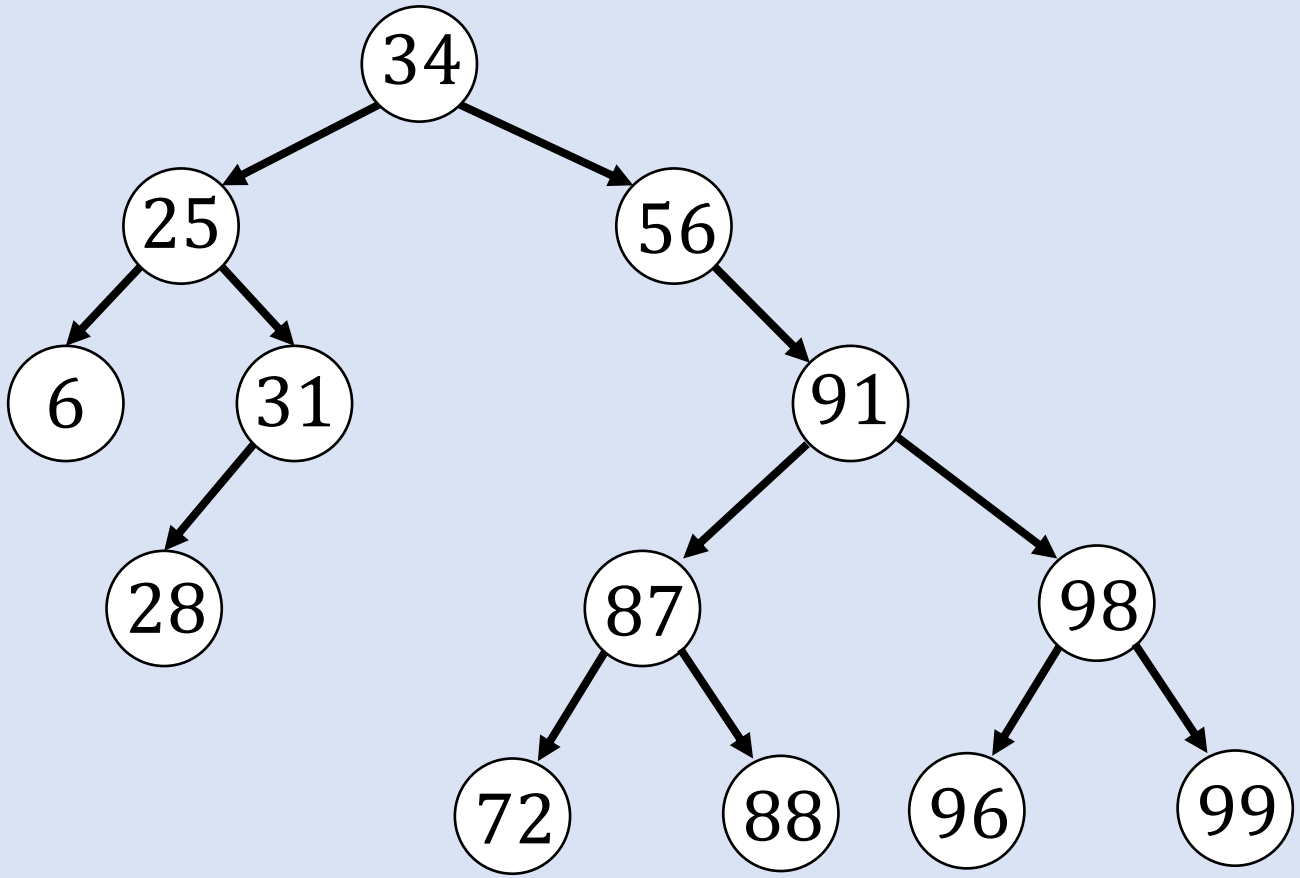


Deletion Scenario 2



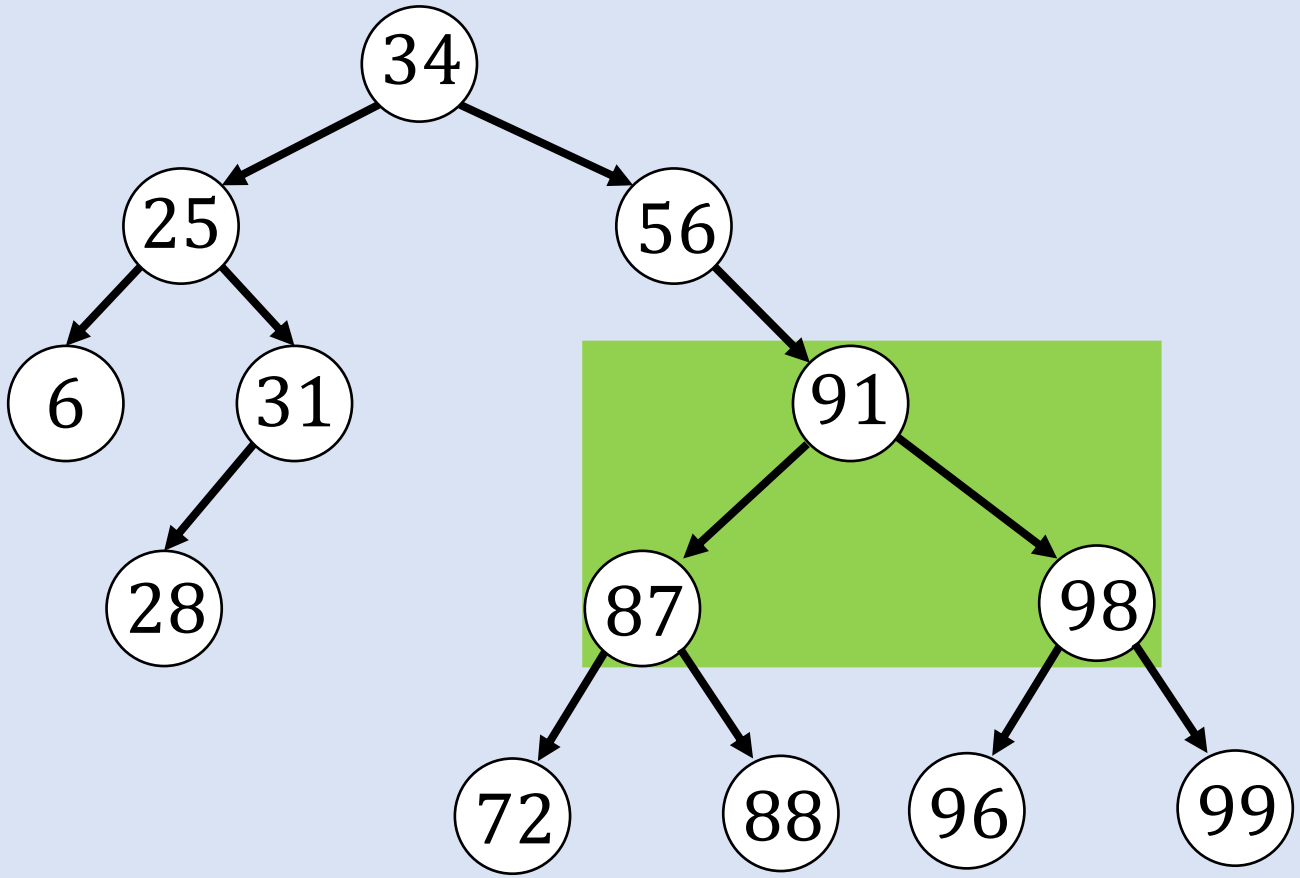
Deletion Scenario 3

Delete the node with value 91.



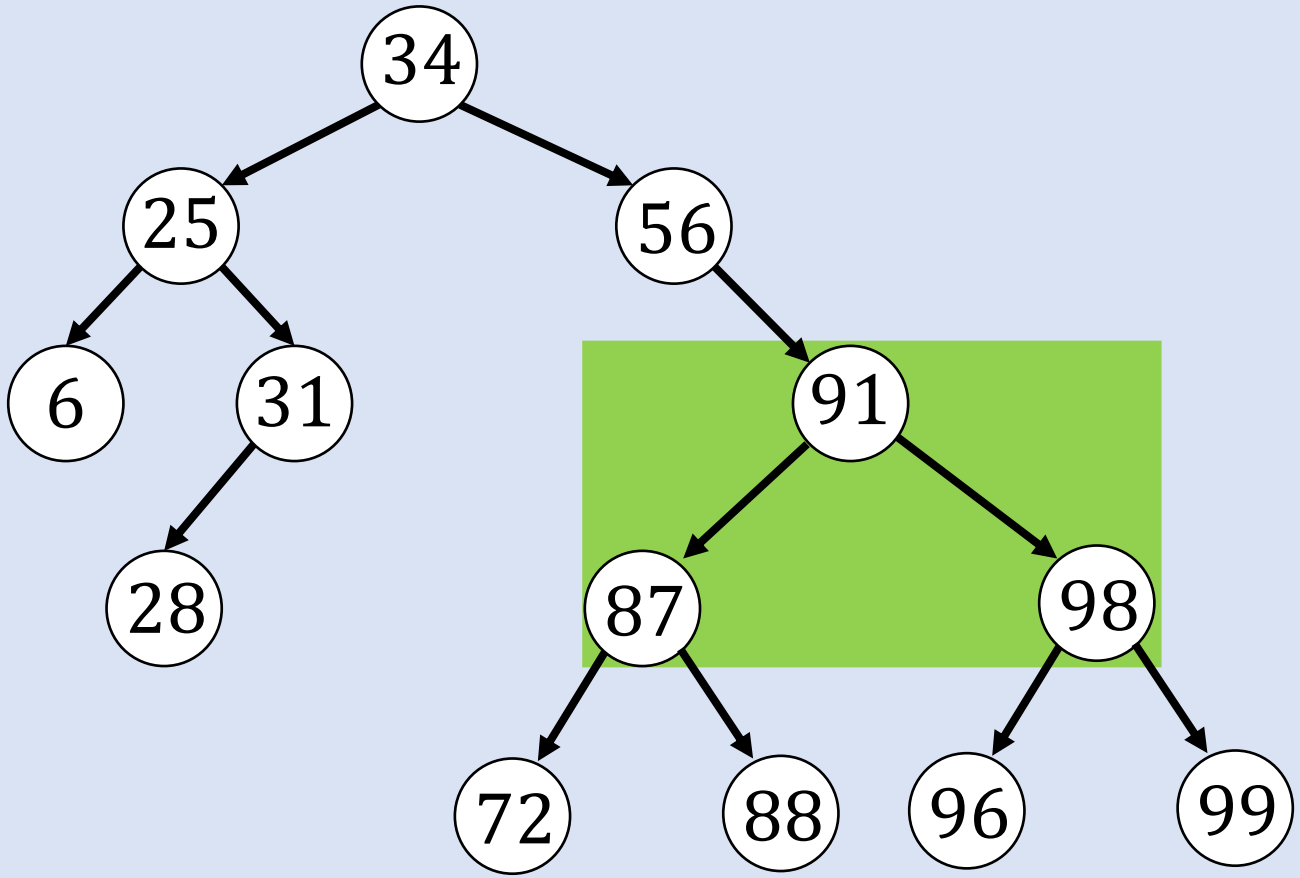
Deletion Scenario 3

91 has only 2 children



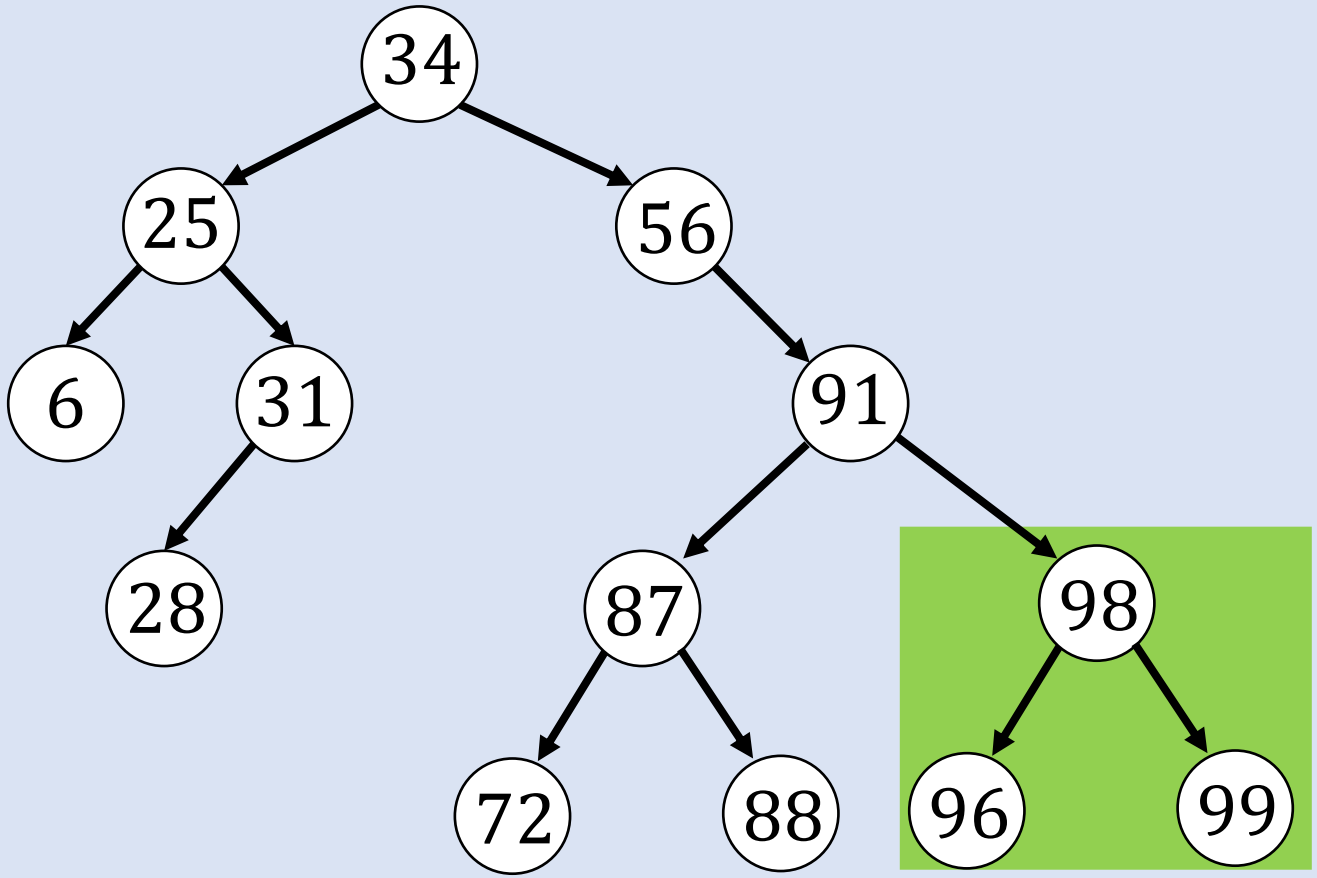
Deletion Scenario 3

Since 91 has two children, we need to pick a proper node to replace 91 while maintaining the BST property.



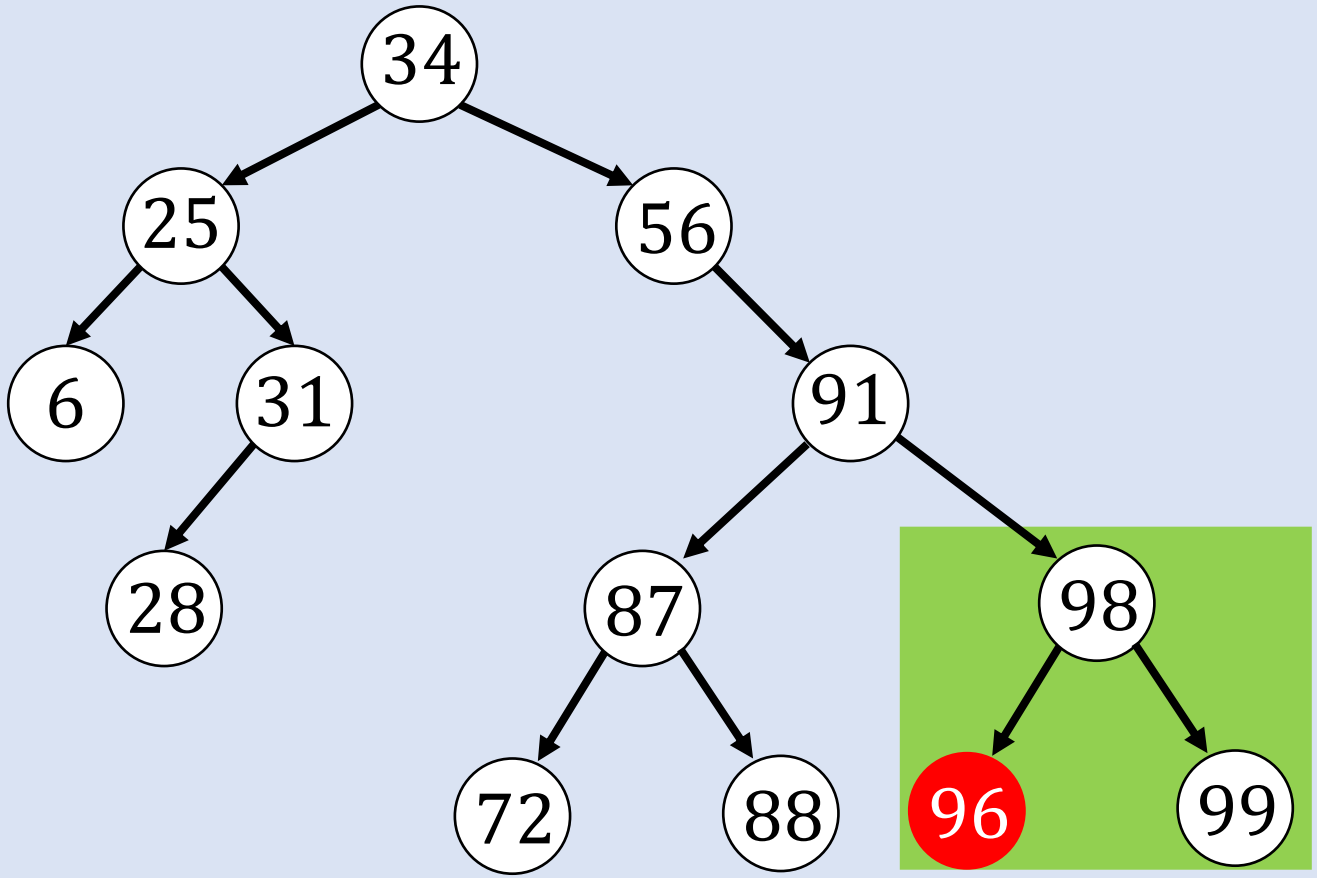
Deletion Scenario 3

We will look for the successor of 91.



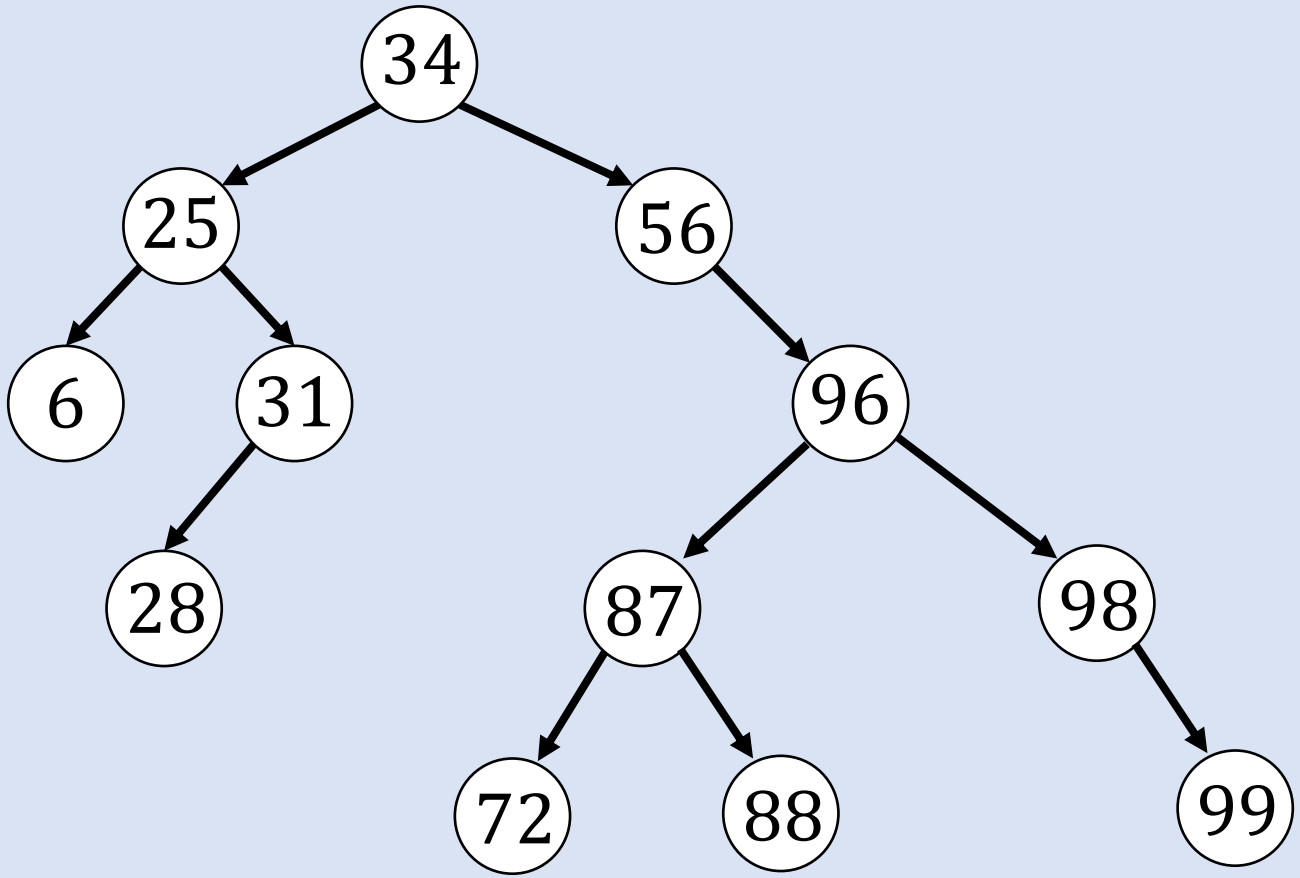
Deletion Scenario 3

The successor of 91 is located at the right subtree. Based on the definition, we should look for the left most node of the subtree. Here, we find that 96 is the successor of 91.

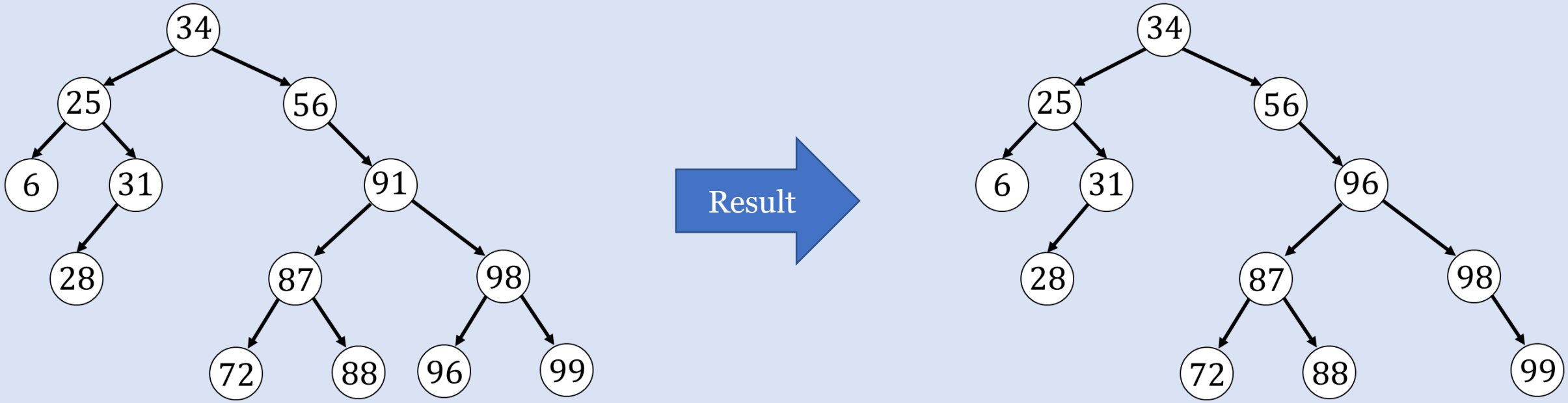


Deletion Scenario 3

The successor replaces the data value that was removed initially.



Deletion Scenario 3



Search

- Since a Binary Search Tree will contain data, we would need to sometimes check to see if the data exists in the structure.
- Traverse the tree to find the respective node.
  - We would start at the root and use the BST property to try and locate the data value in the structure.
  - If the value we are searching for is smaller than the current value we are at, then we would traverse to the left child.
  - If the value we are searching for is larger than the current value we are at, then we would traverse to the right child.
- If we reach a leaf and have not found the data, then we can say that the item does not belong into the structure.
- Runtime:
  - Worst-Case: O(n)
  - Average-Case: O(h)
  - Best-Case: O(h)

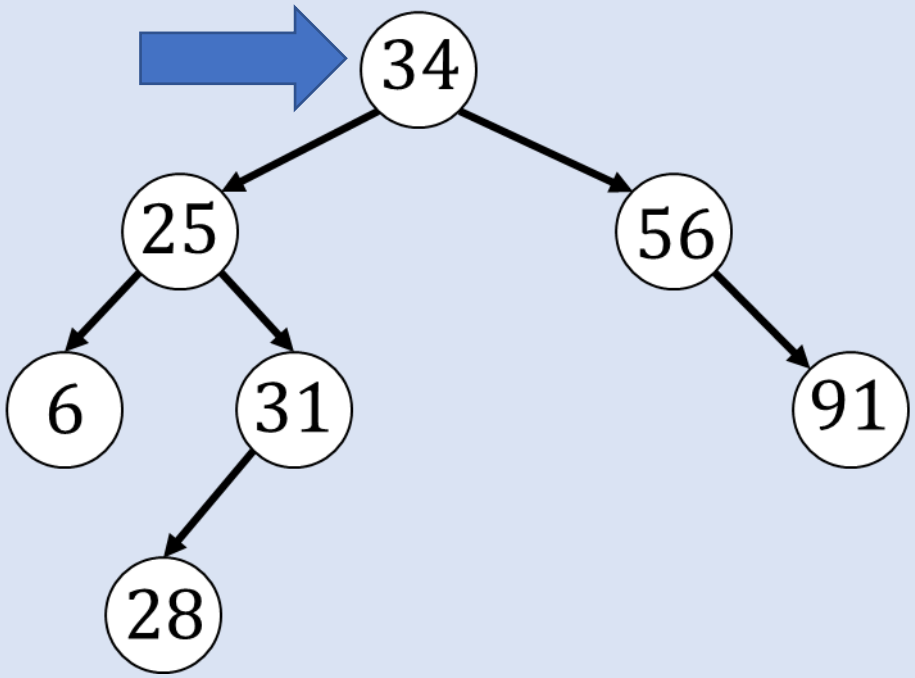
Traversal

- There are three ways to traverse a tree and display all values stored.
- Preorder
  - Root, Left, Right
- Postorder
  - Left, Right, Root
- Inorder
  - Left, Root, Right

Inorder Traverse

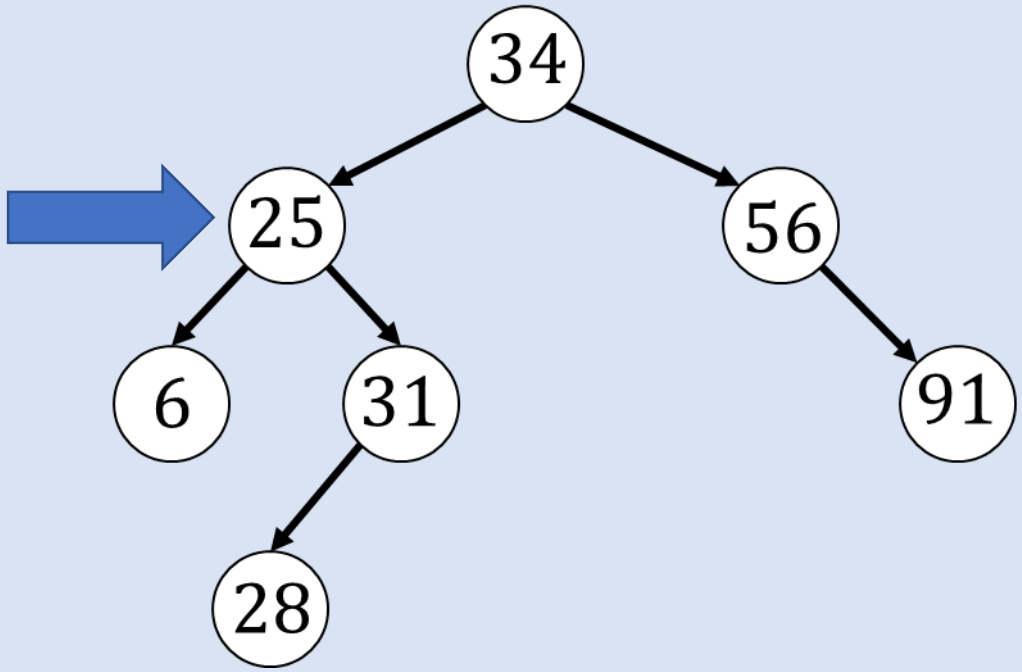
- Inorder traversal can properly print the nodes in ascending order through the following steps.
- The steps an inorder traversal takes are:
  - Traverse the left subtree inorder
  - Process/Print the value
  - Traverse the right subtree inorder

Inorder Traversal Example



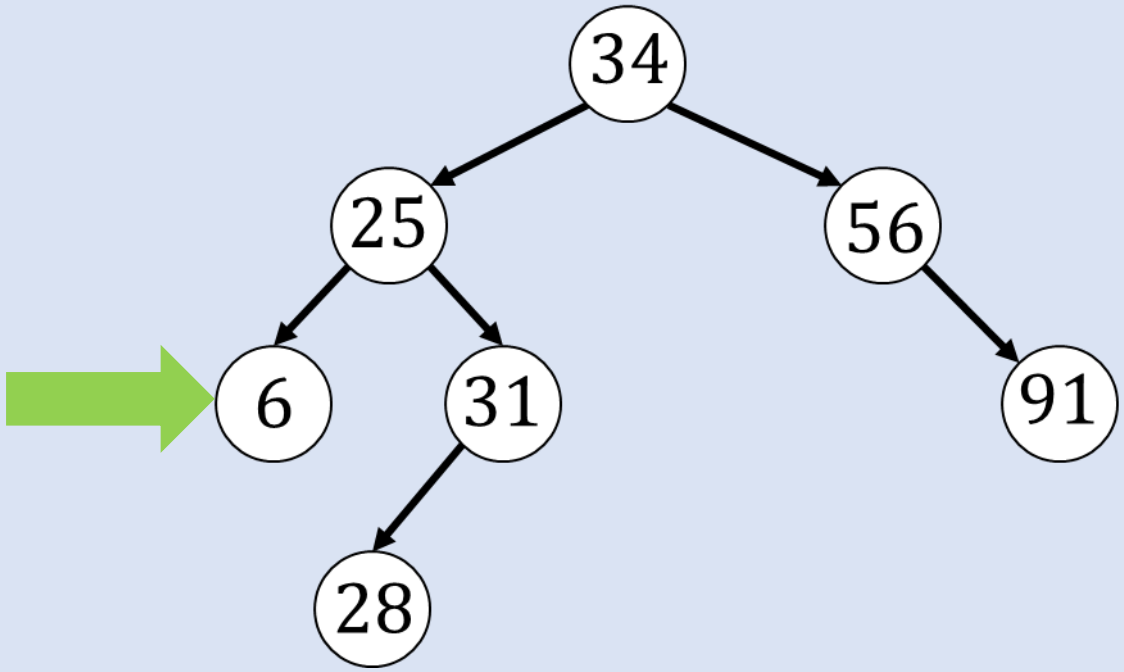
Data Value Processed

Inorder Traversal Example



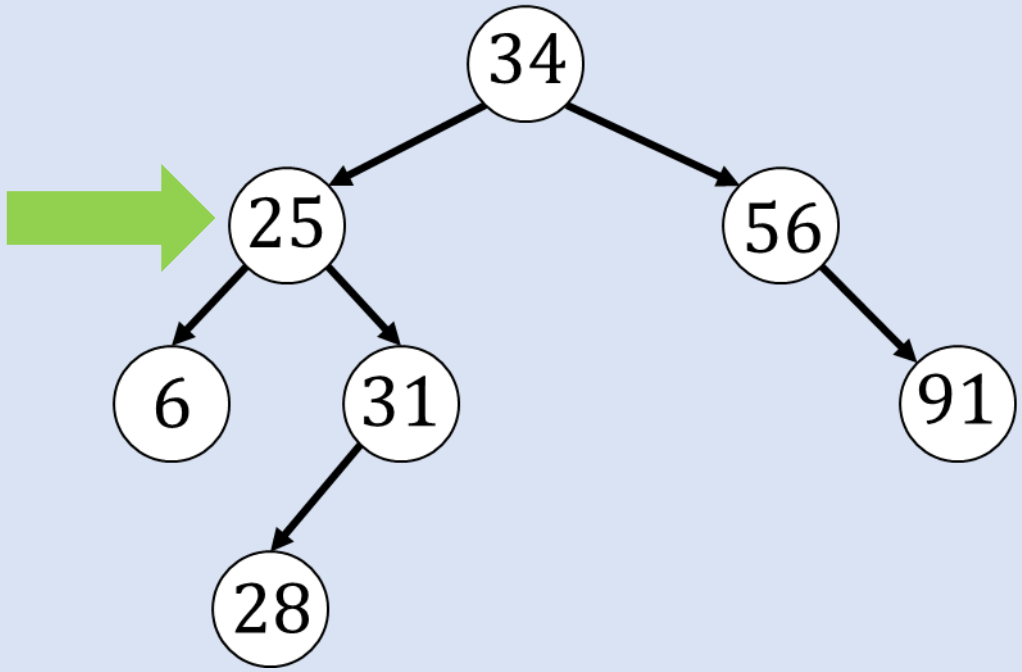
Data Value Processed

Inorder Traversal Example



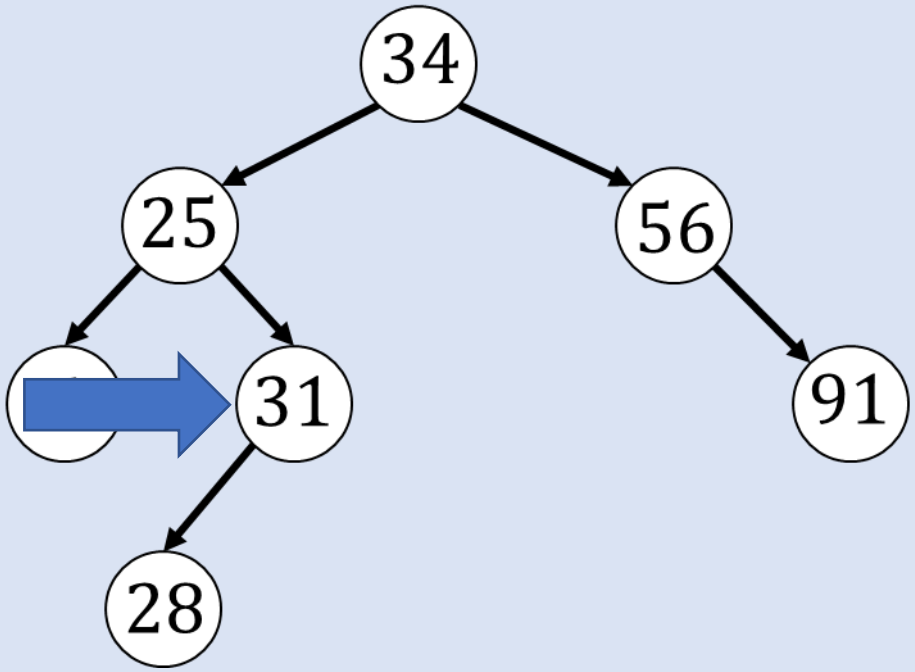
Data Value Processed
6

Inorder Traversal Example



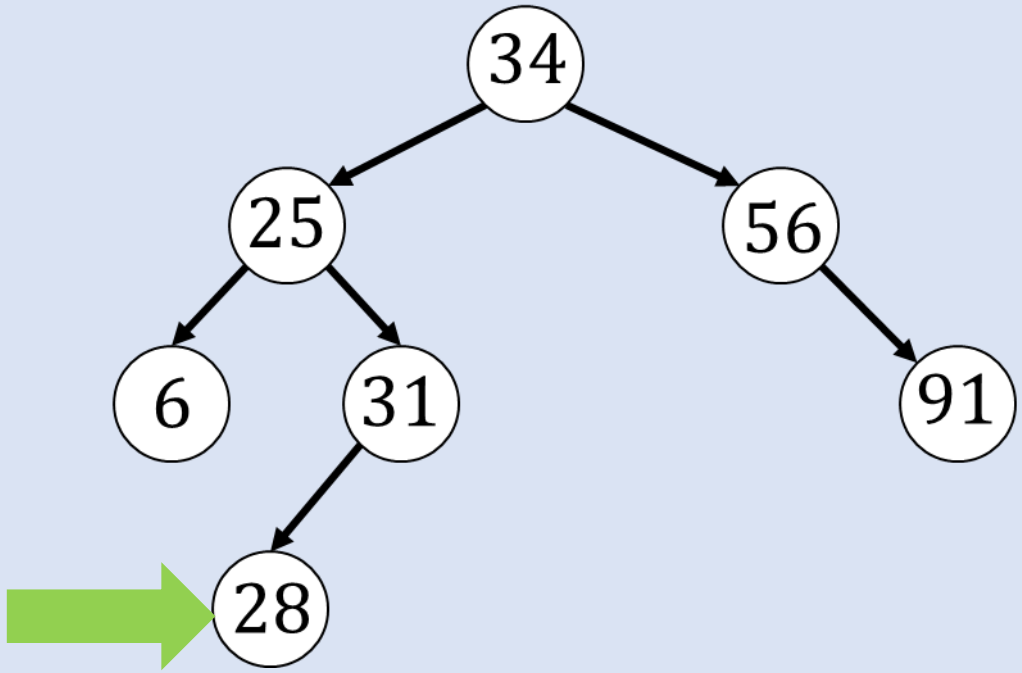
Data Value Processed
6
25

Inorder Traversal Example



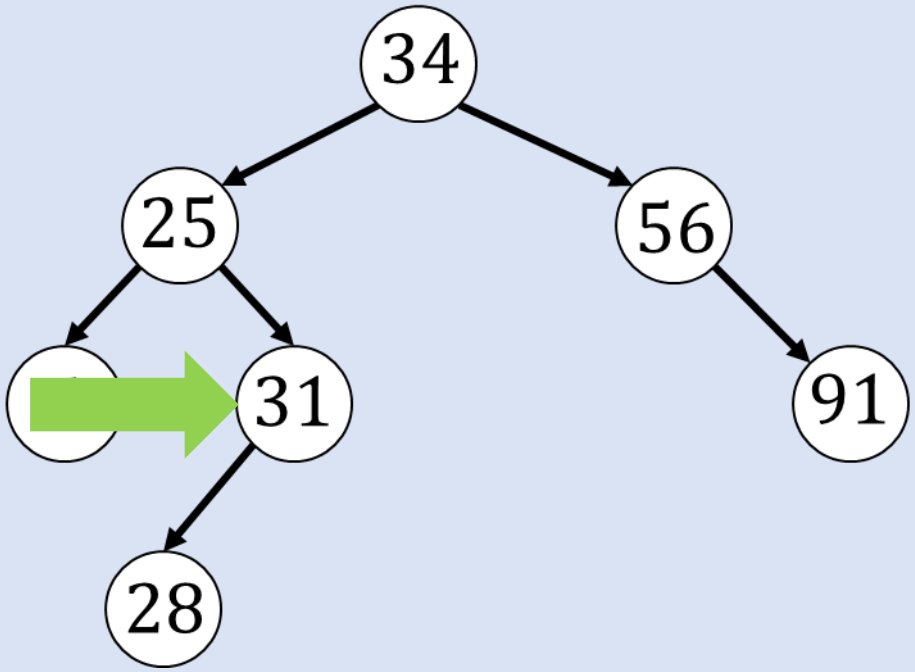
Data Value Processed
6
25

Inorder Traversal Example



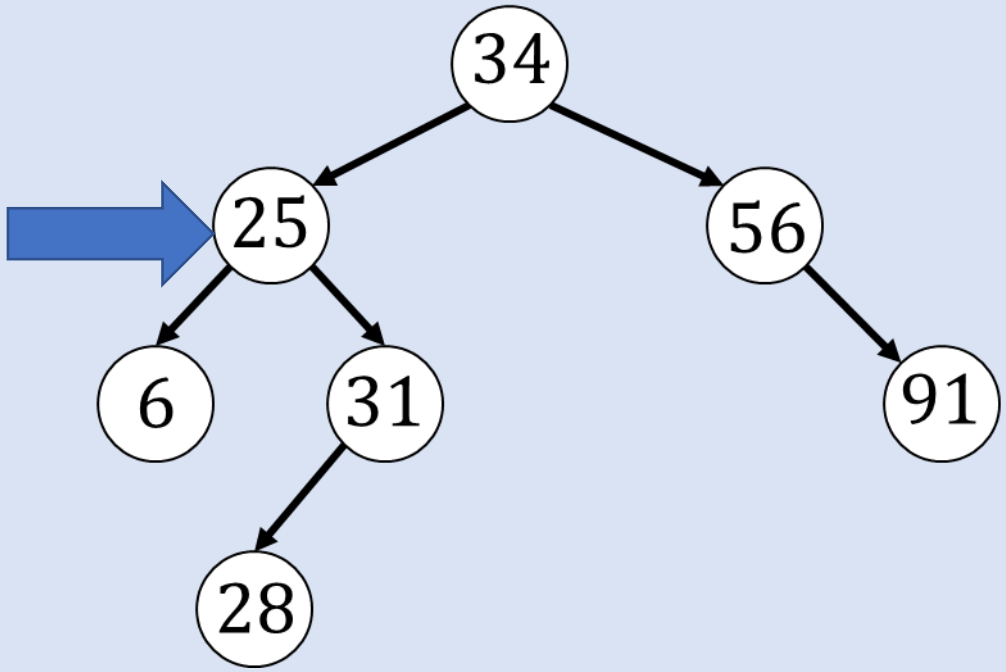
Data Value Processed
6
25
28

Inorder Traversal Example



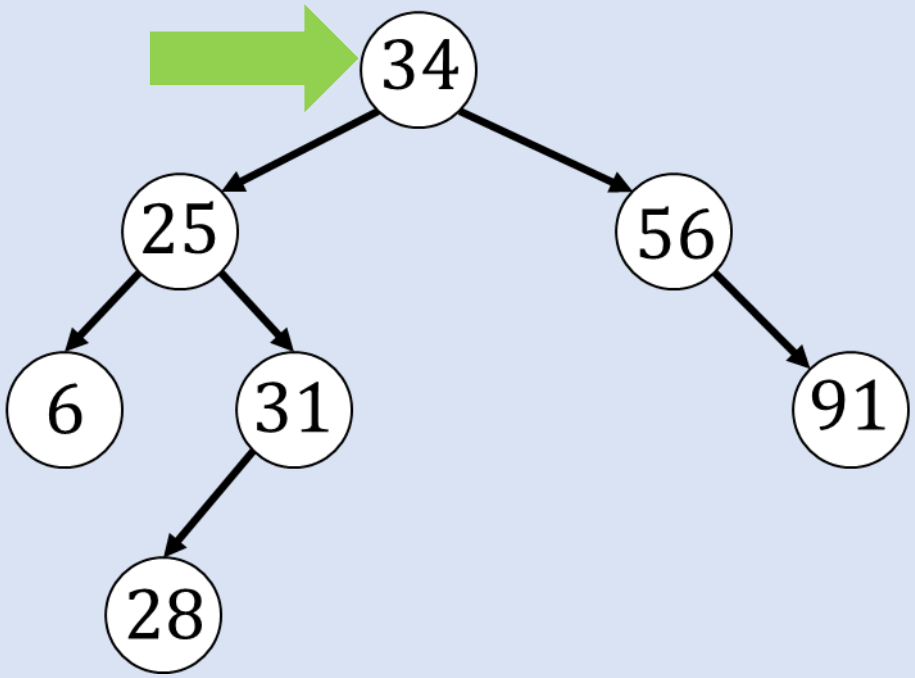
Data Value Processed
6
25
28
31

Inorder Traversal Example



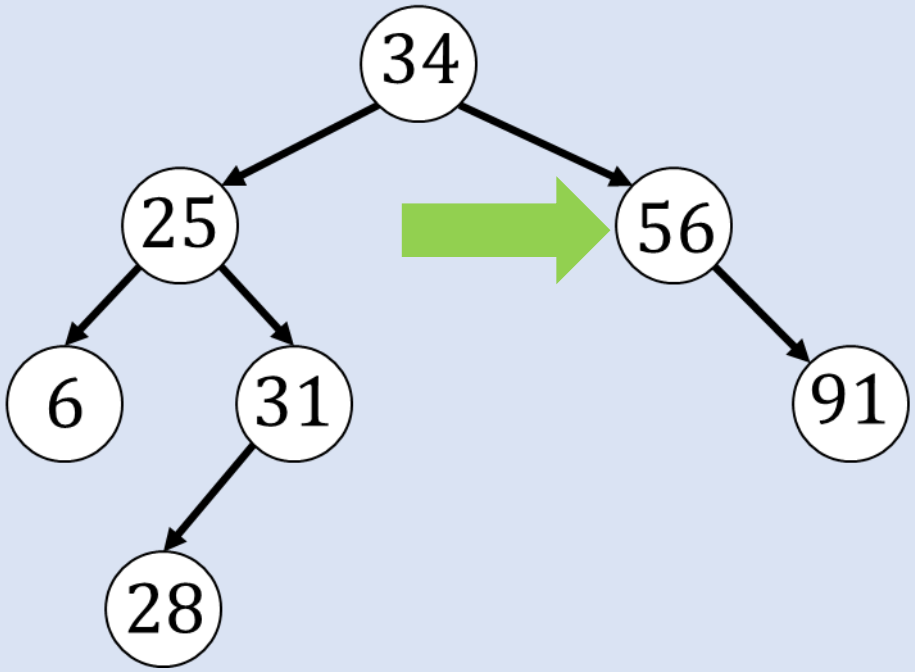
Data Value Processed
6
25
28
31

Inorder Traversal Example



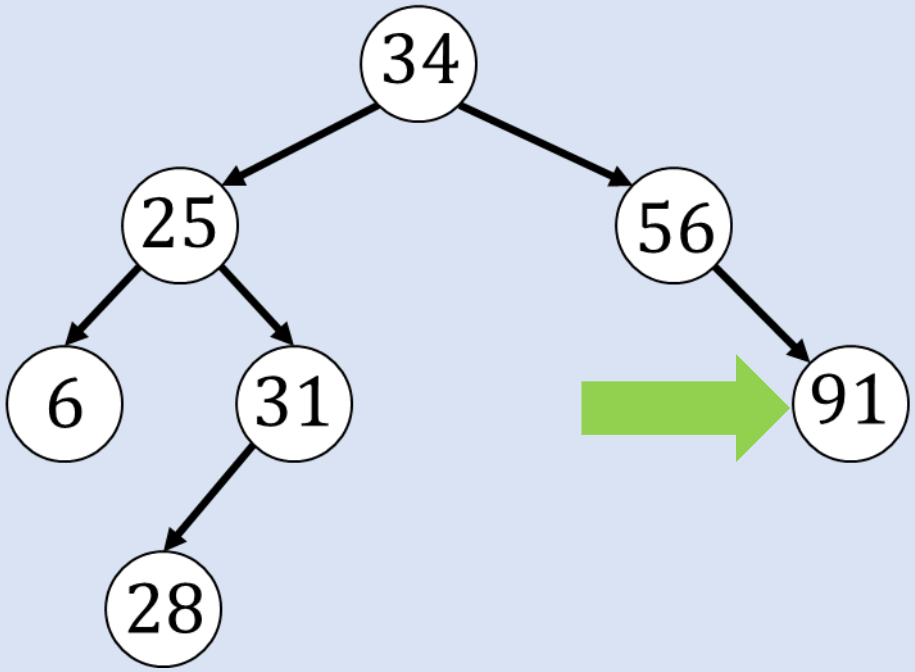
Data Value Processed
6
25
28
31
34

Inorder Traversal Example



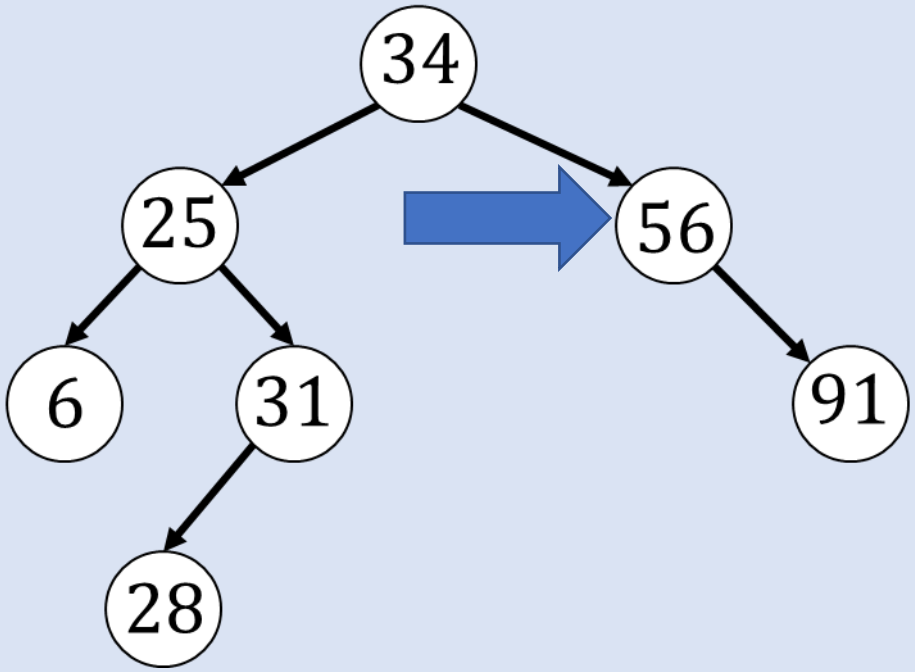
Data Value Processed
6
25
28
31
34
56

Inorder Traversal Example



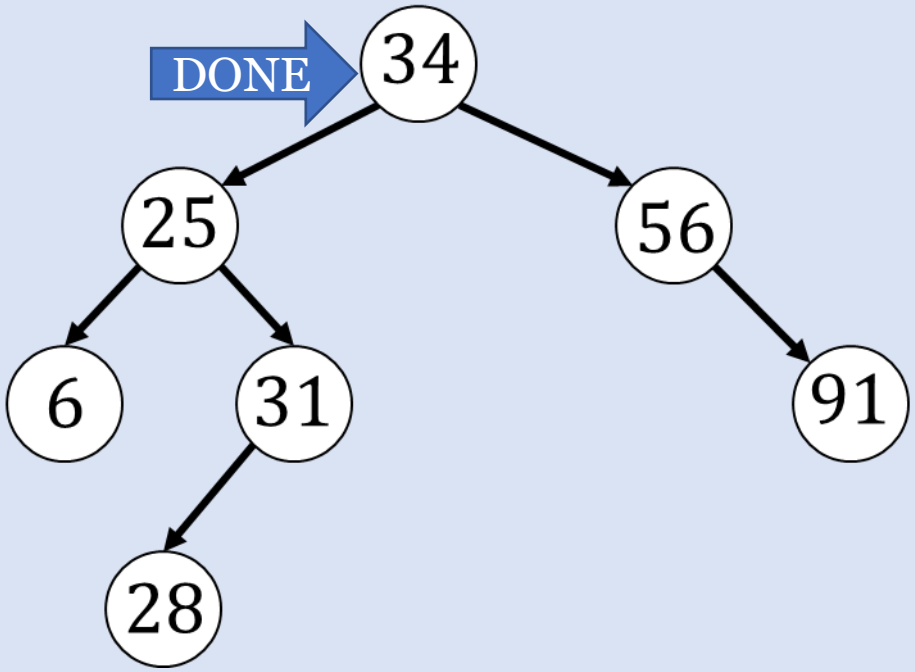
Data Value Processed
6
25
28
31
34
56
91

Inorder Traversal Example



Data Value Processed
6
25
28
31
34
56
91

Inorder Traversal Example

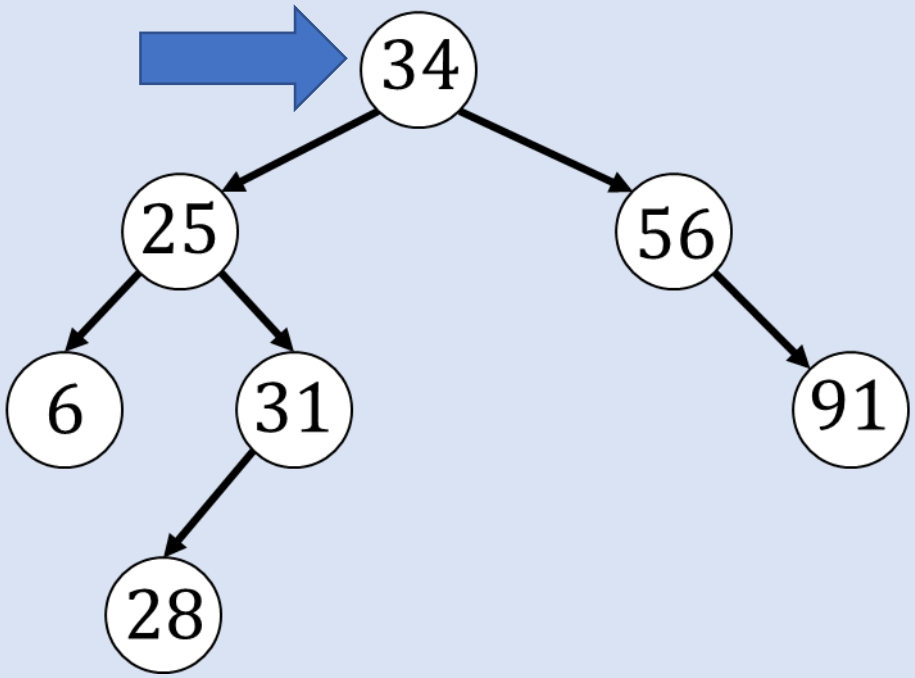


Data Value Processed
6
25
28
31
34
56
91

Preorder Traverse

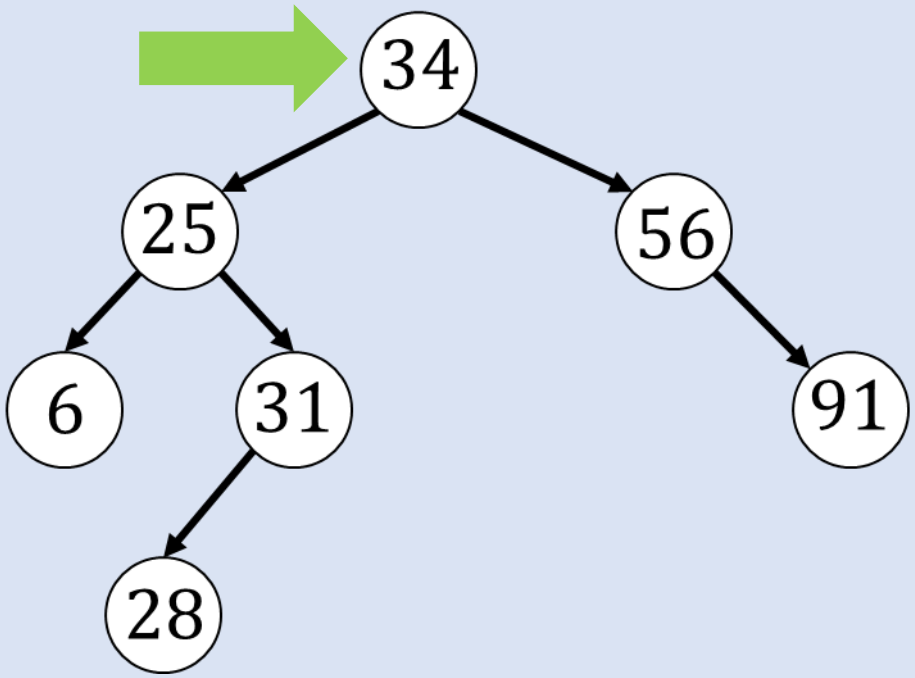
- Preorder traversal is very useful if you wanted to make a duplicate copy of the tree with the same exact structure.
- The steps a preorder traversal takes are:
  - Process/Print the value
  - Traverse the left subtree preorder
  - Traverse the right subtree preorder

Preorder Traverse Example



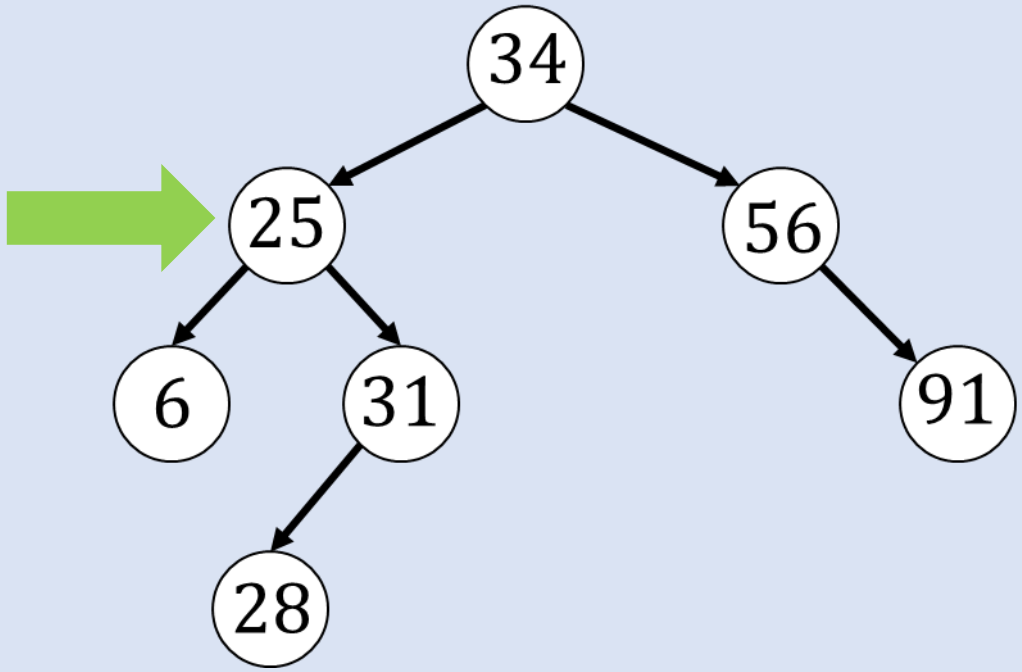
Data Value Processed

Preorder Traverse Example



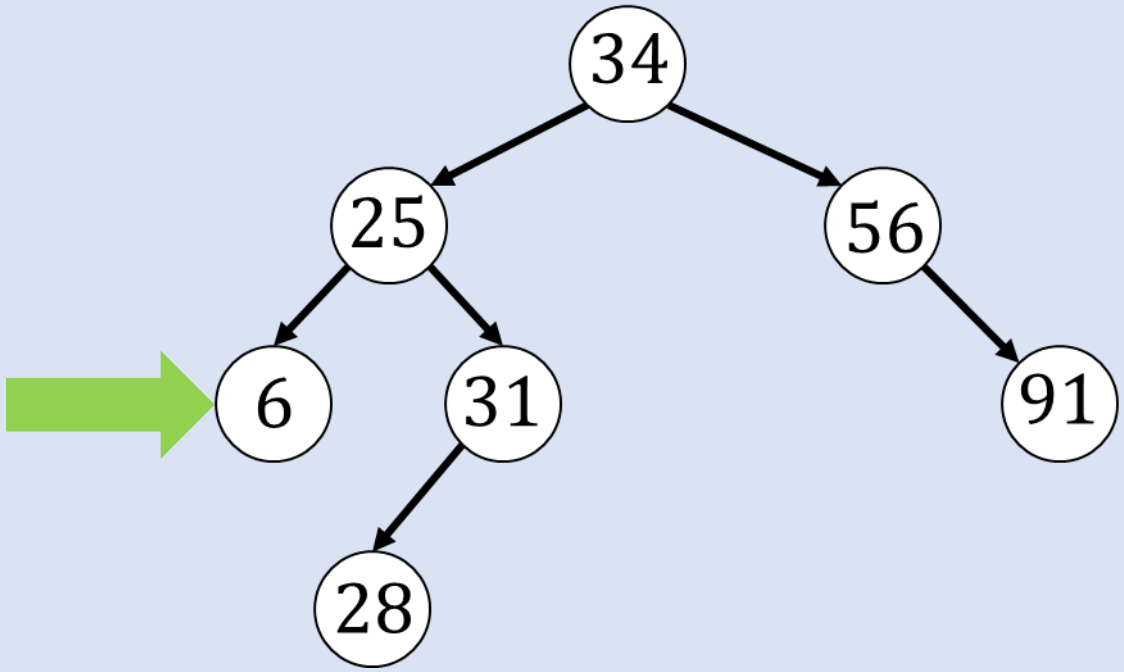
Data Value Processed
34

Preorder Traverse Example



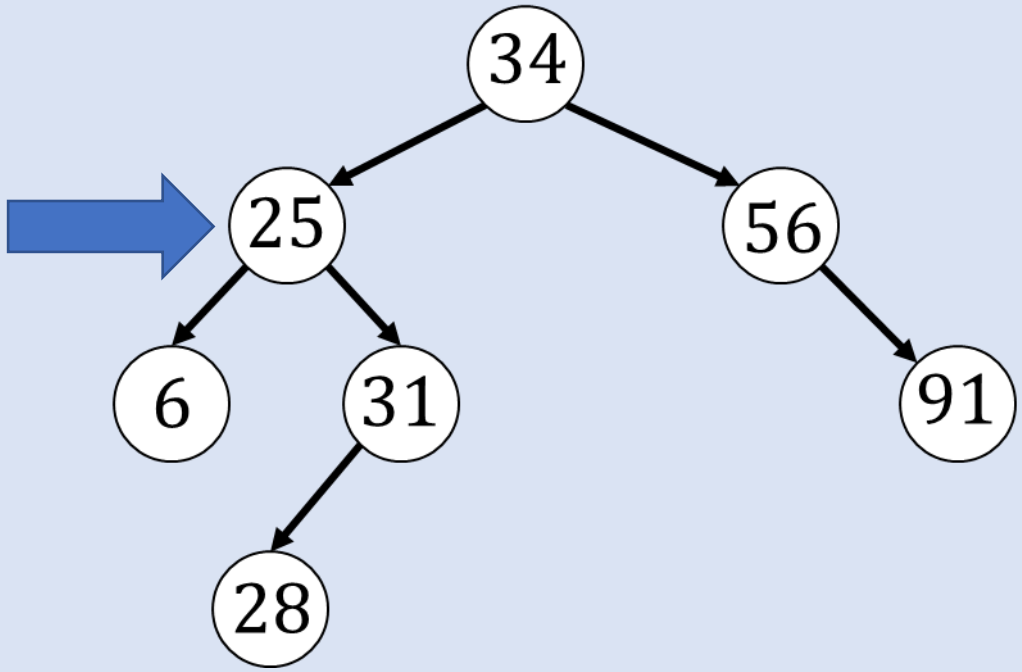
Data Value Processed
34
25

Preorder Traverse Example



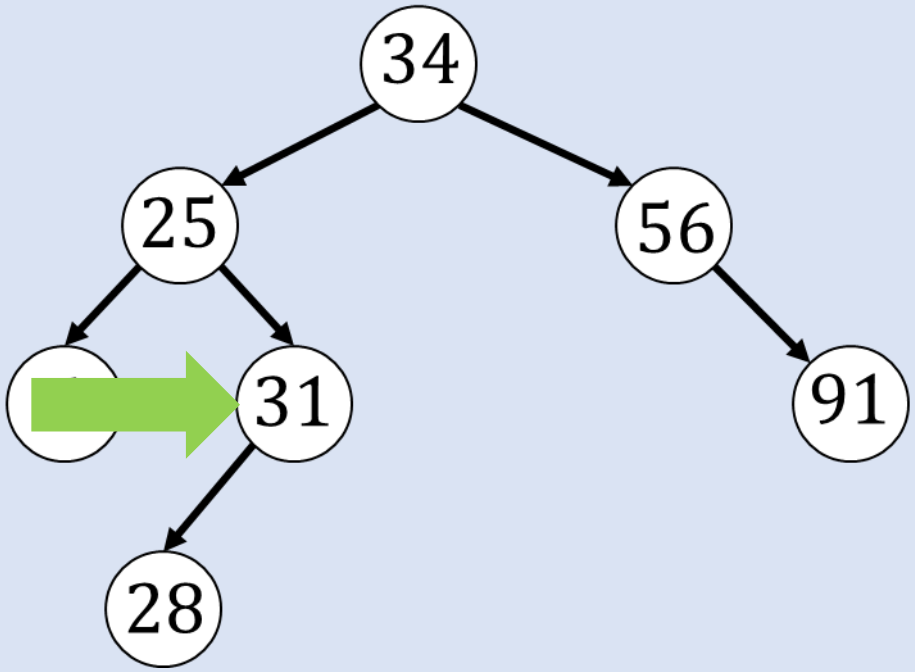
Data Value Processed
34
25
6

Preorder Traverse Example



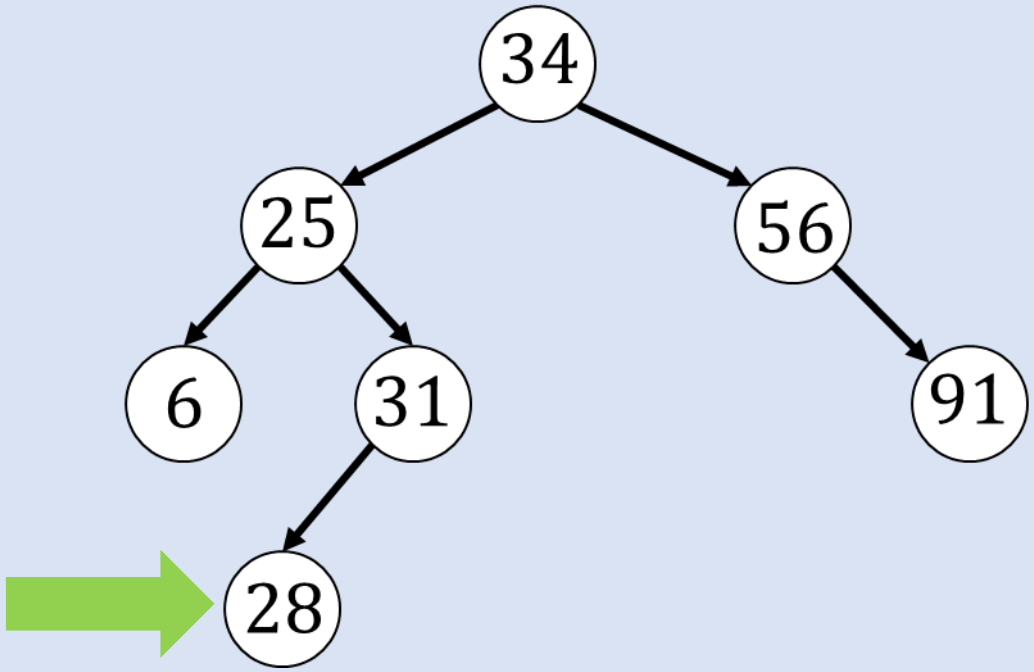
Data Value Processed
34
25
6

Preorder Traverse Example



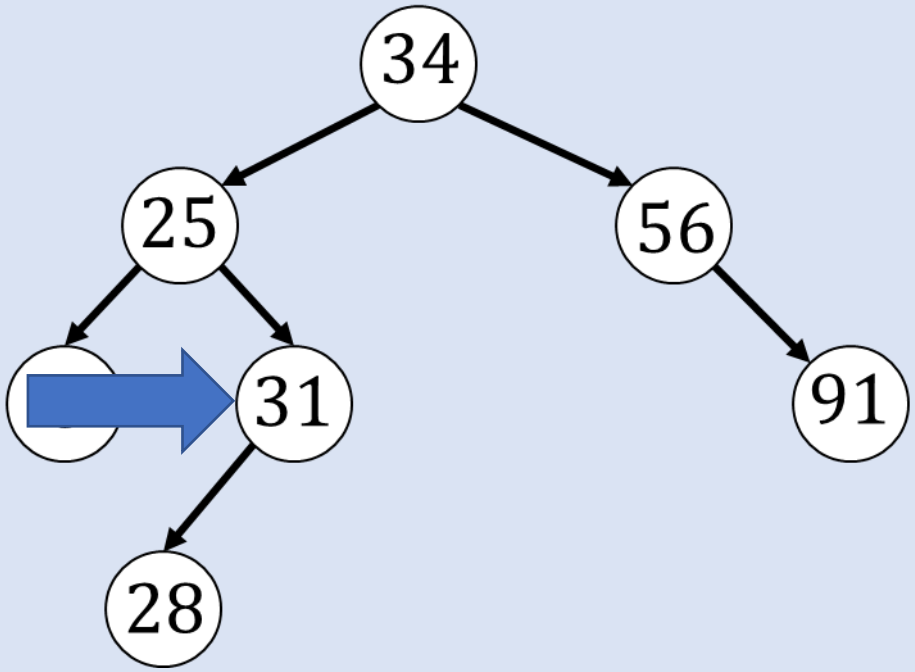
Data Value Processed
34
25
6
31

Preorder Traverse Example



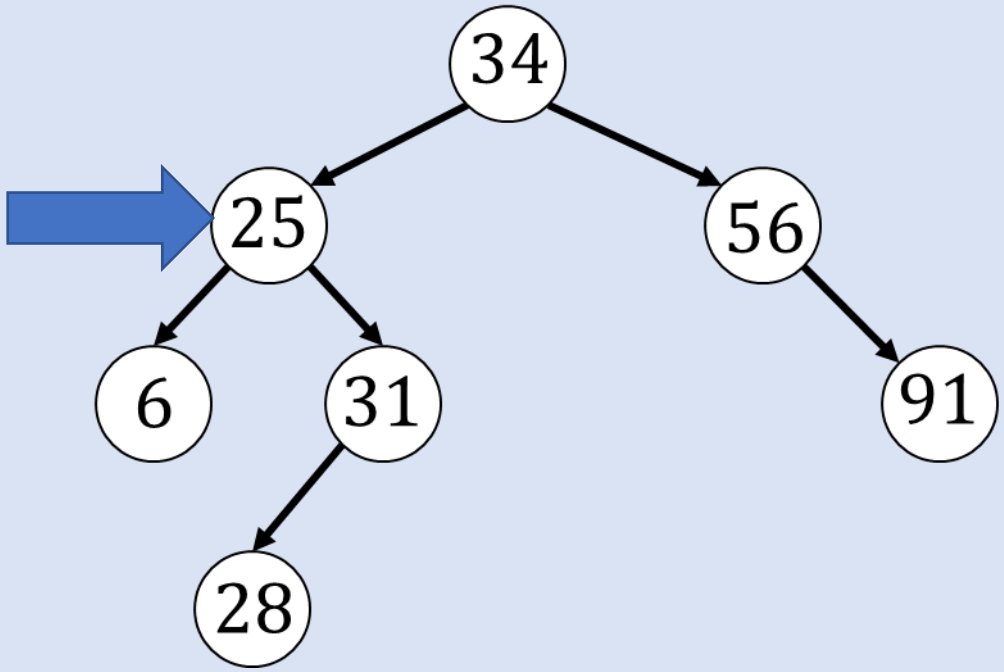
Data Value Processed
34
25
6
31
28

Preorder Traverse Example



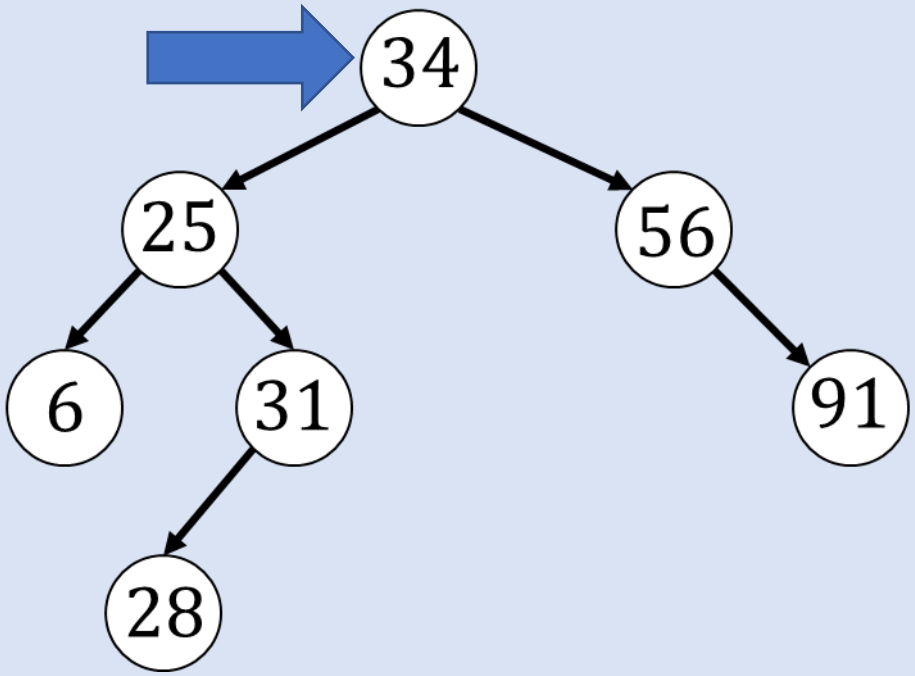
Data Value Processed
34
25
6
31
28

Preorder Traverse Example



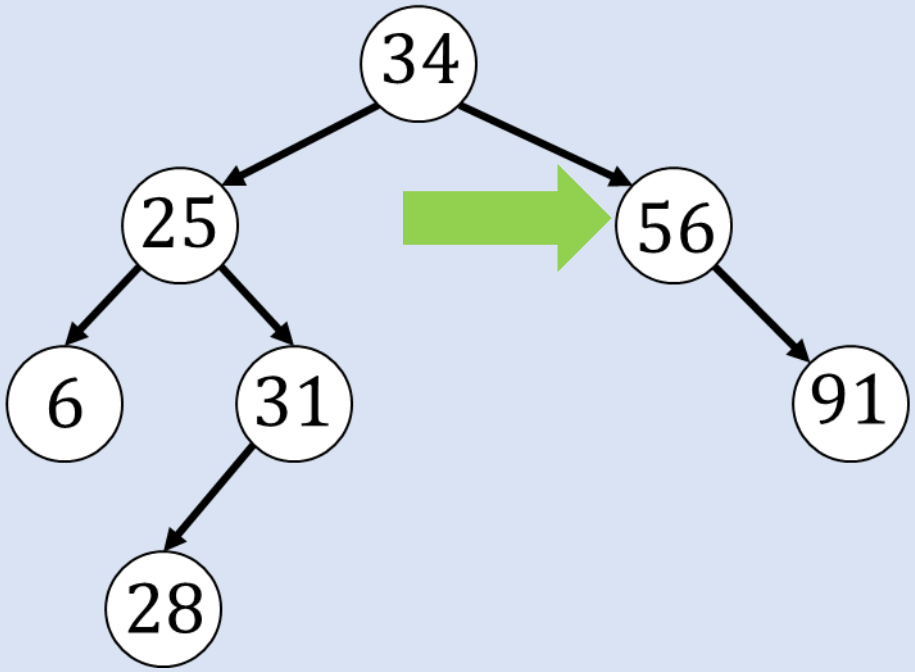
Data Value Processed
34
25
6
31
28

Preorder Traverse Example



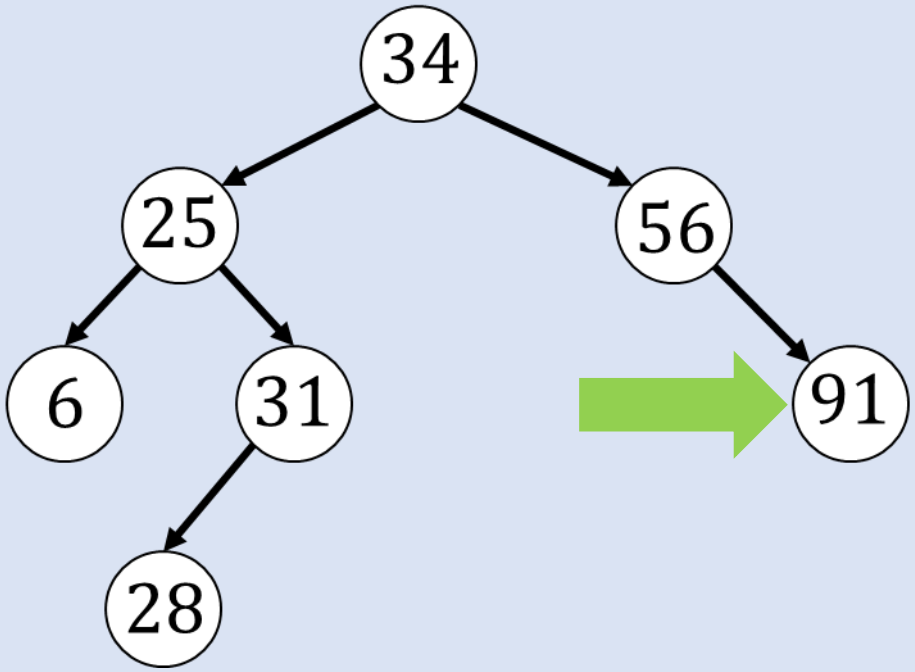
Data Value Processed
34
25
6
31
28

Preorder Traverse Example



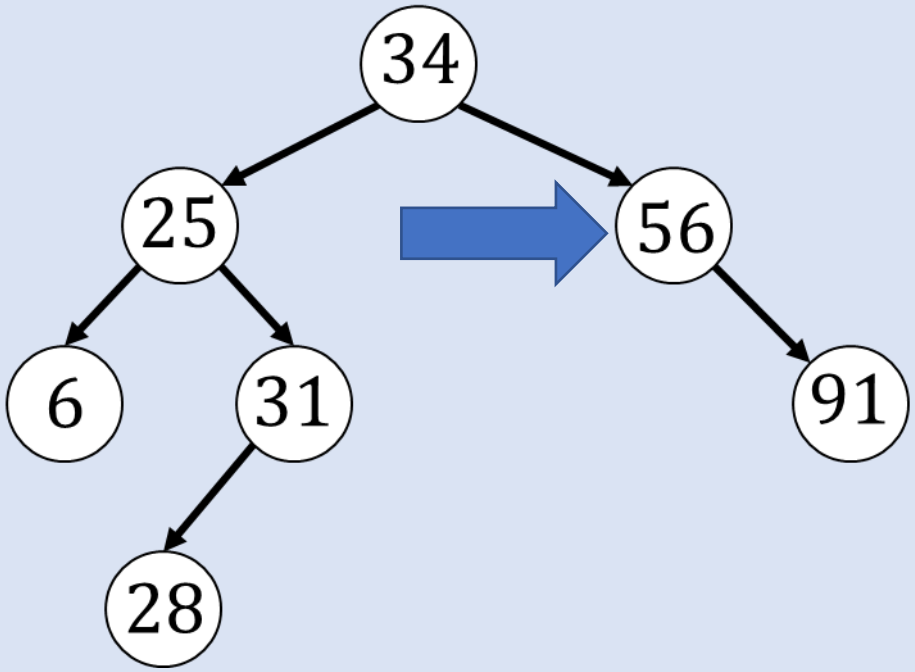
Data Value Processed
34
25
6
31
28
56

Preorder Traverse Example



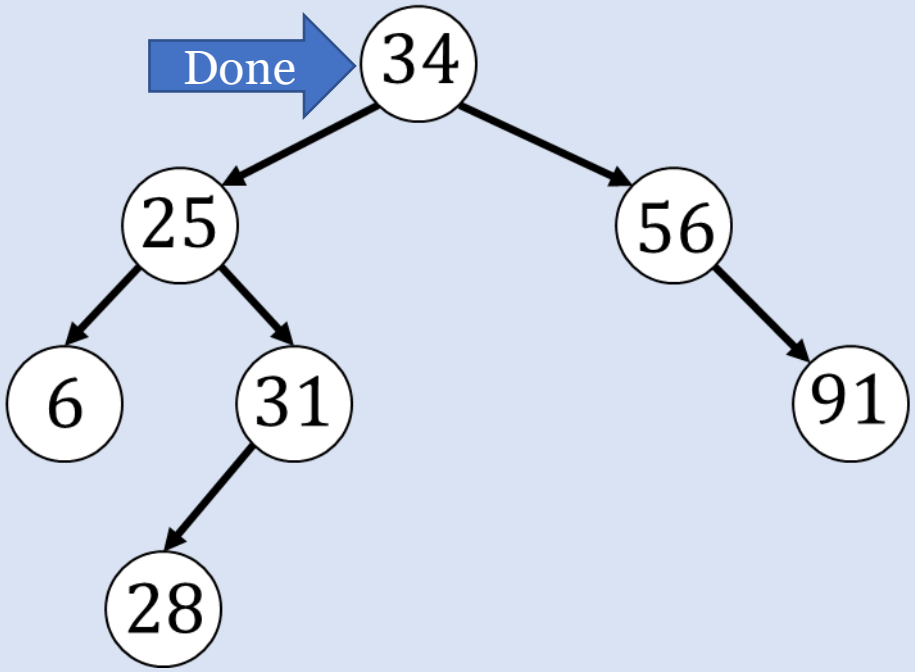
Data Value Processed
34
25
6
31
28
56
91

Preorder Traverse Example



Data Value Processed
34
25
6
31
28
56
91

Preorder Traverse Example

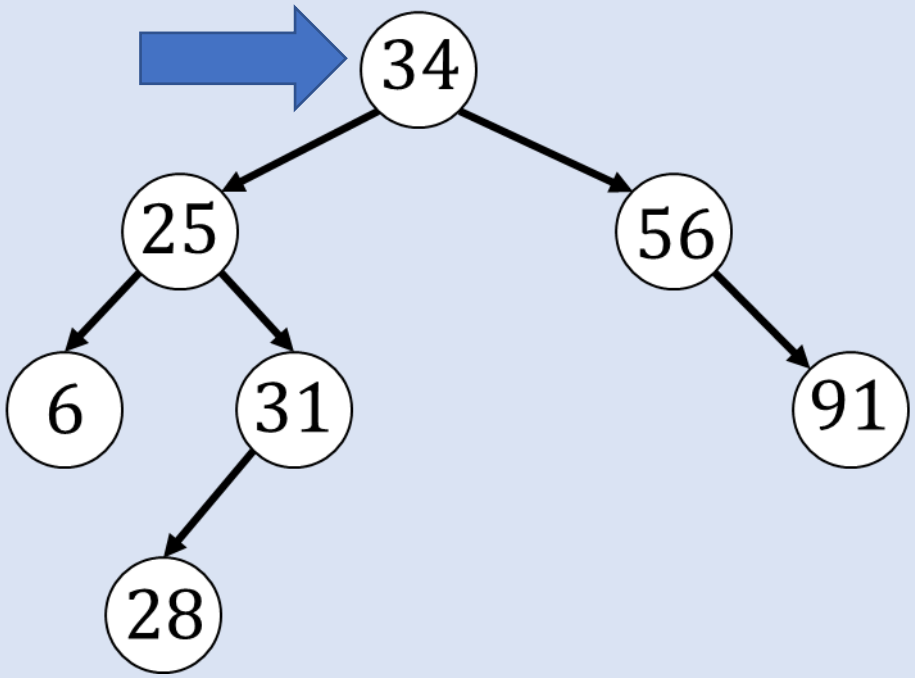


Data Value Processed
34
25
6
31
28
56
91

Postorder Traverse

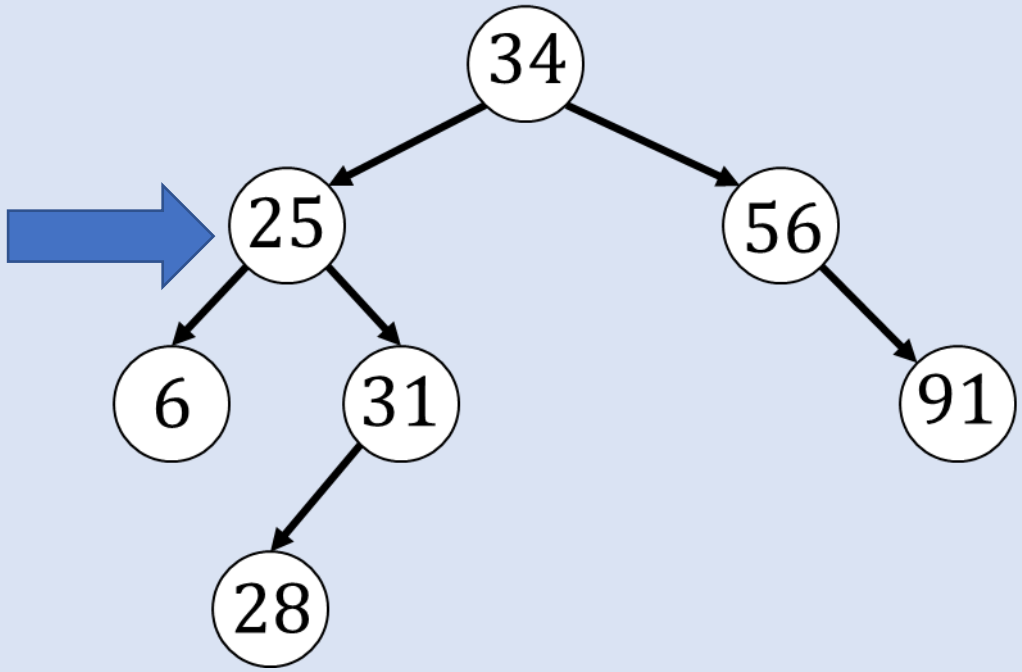
- Postorder traversal is very useful if you wanted to properly delete tree. This would prevent memory leaks along with segmentation faults.
- The steps a postorder traversal takes are:
  - Traverse the left subtree postorder
  - Traverse the right subtree postorder
  - Process/Print the value

Postorder Traverse Example



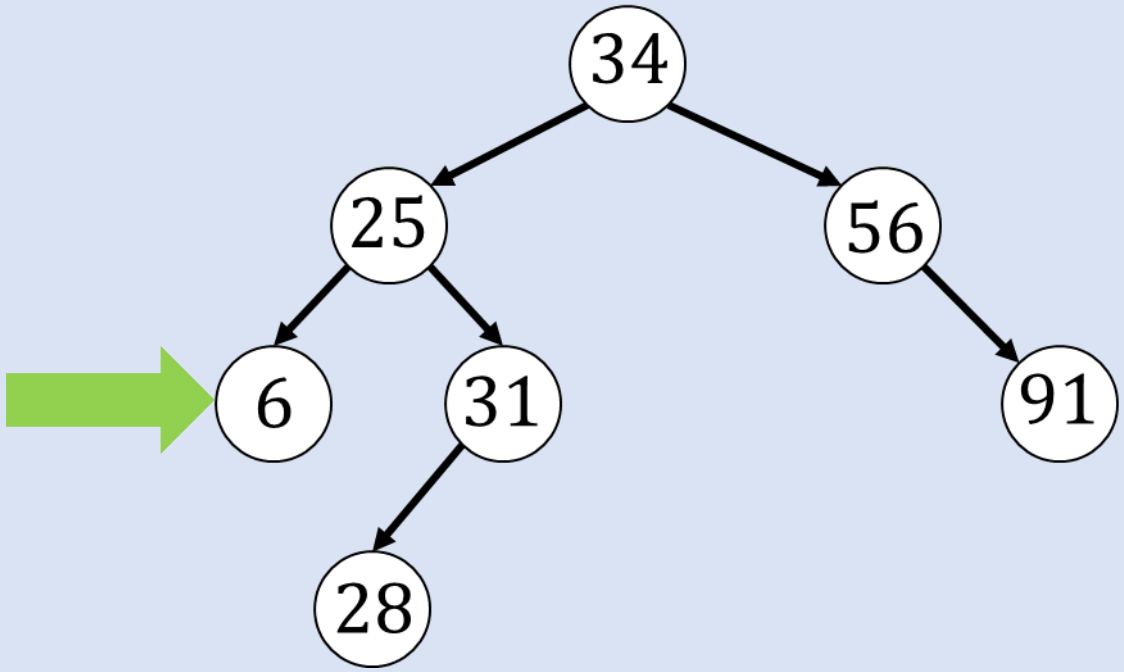
Data Value Processed

Postorder Traverse Example



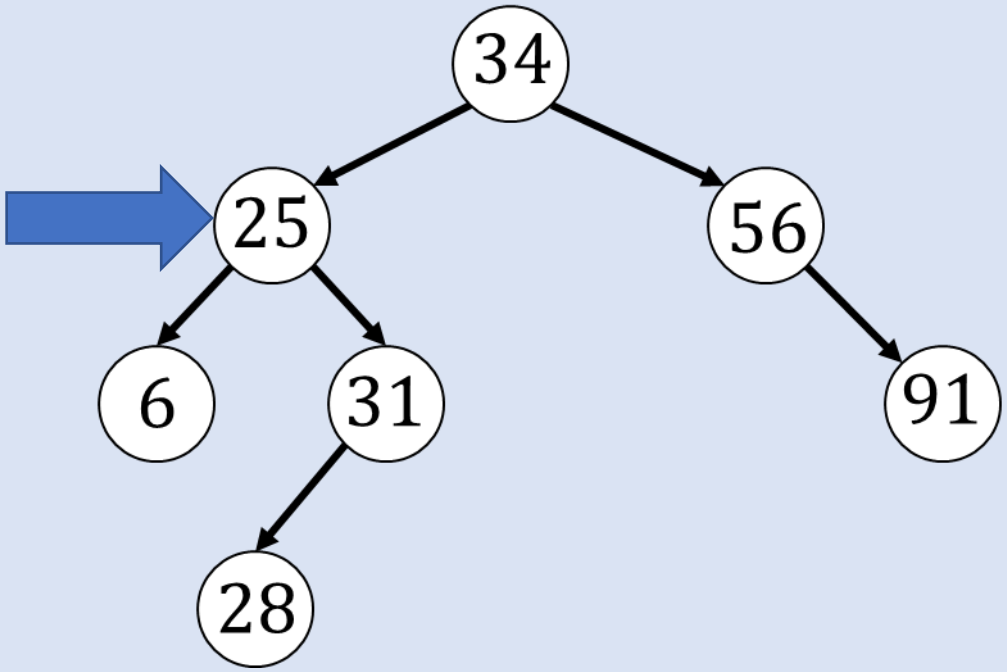
Data Value Processed

Postorder Traverse Example



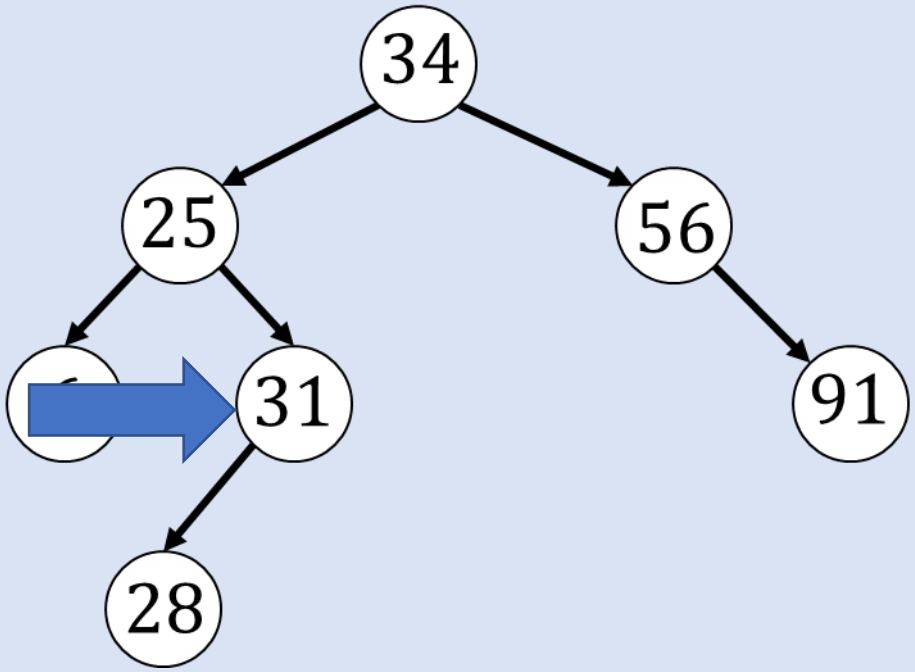
Data Value Processed
6

Postorder Traverse Example



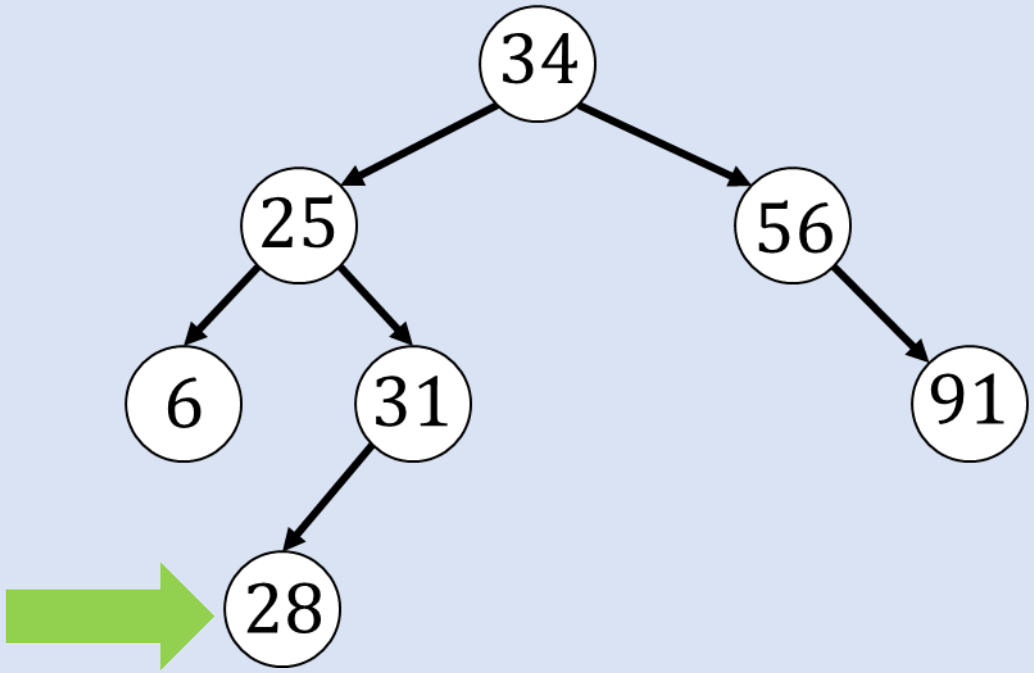
Data Value Processed
6

Postorder Traverse Example



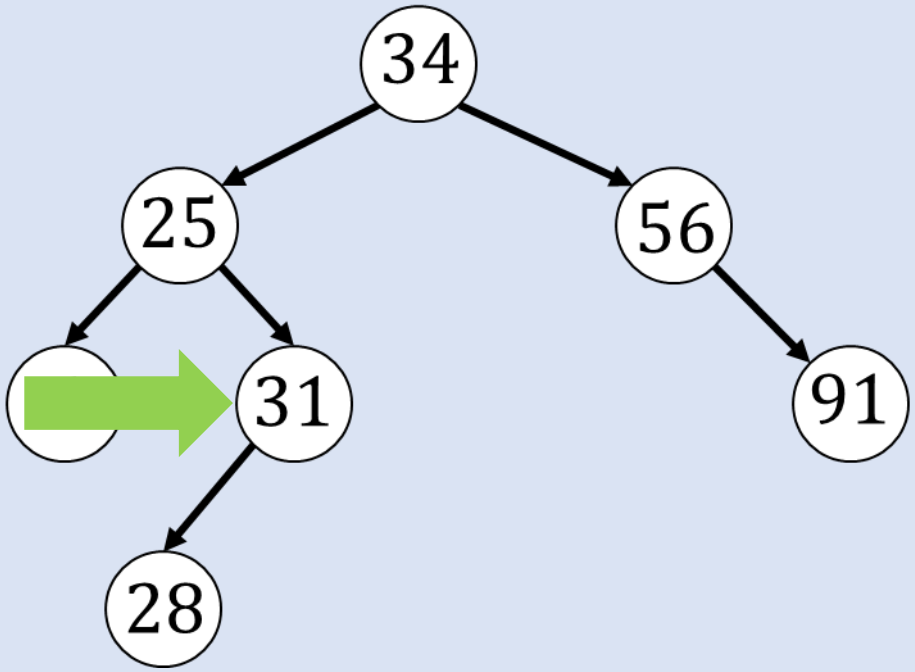
Data Value Processed
6

Postorder Traverse Example



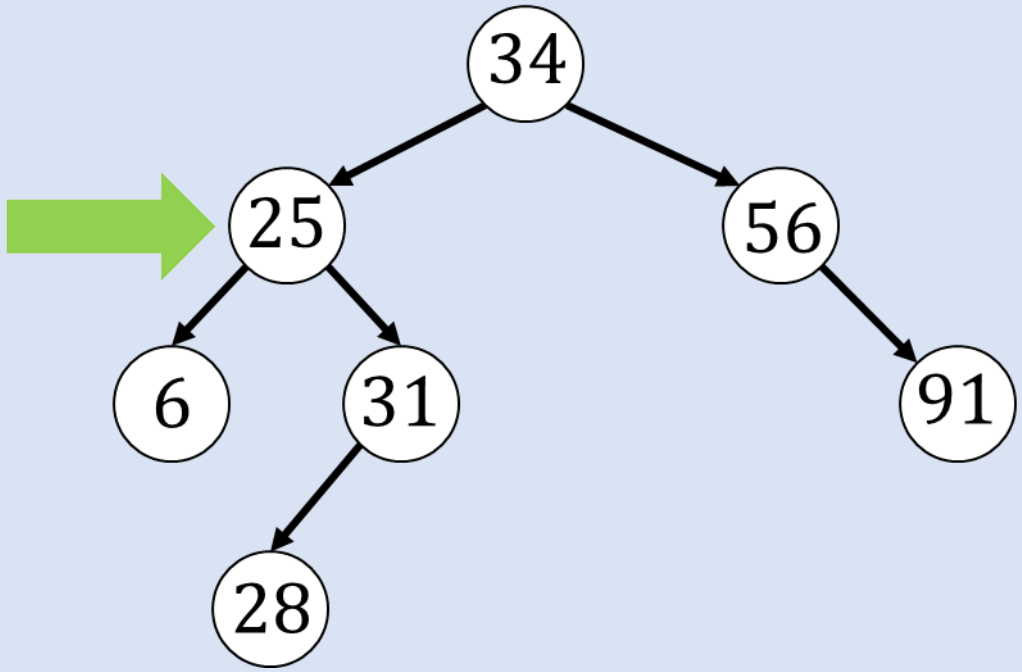
Data Value Processed
6
28

Postorder Traverse Example



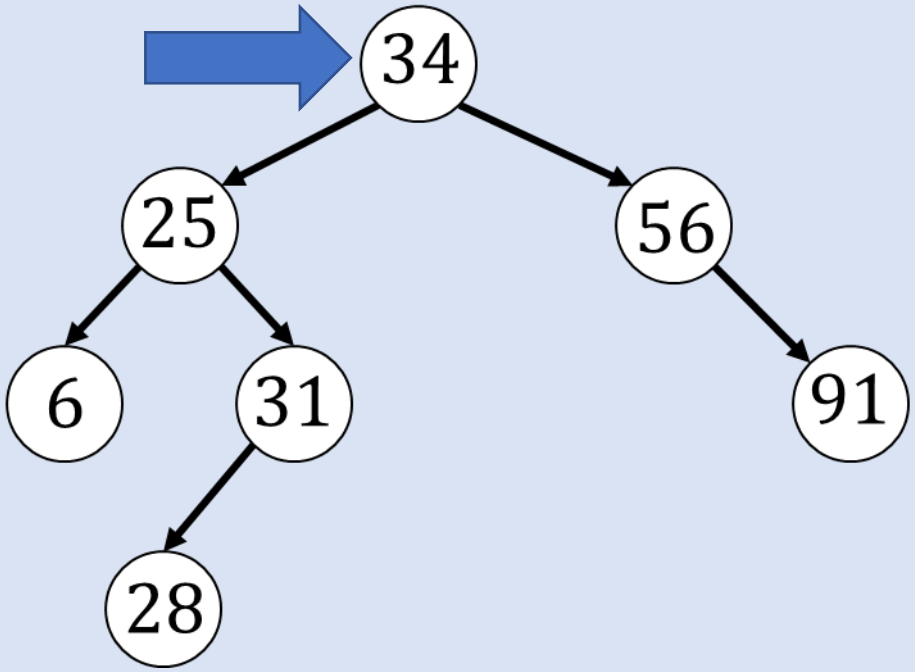
Data Value Processed
6
28
31

Postorder Traverse Example



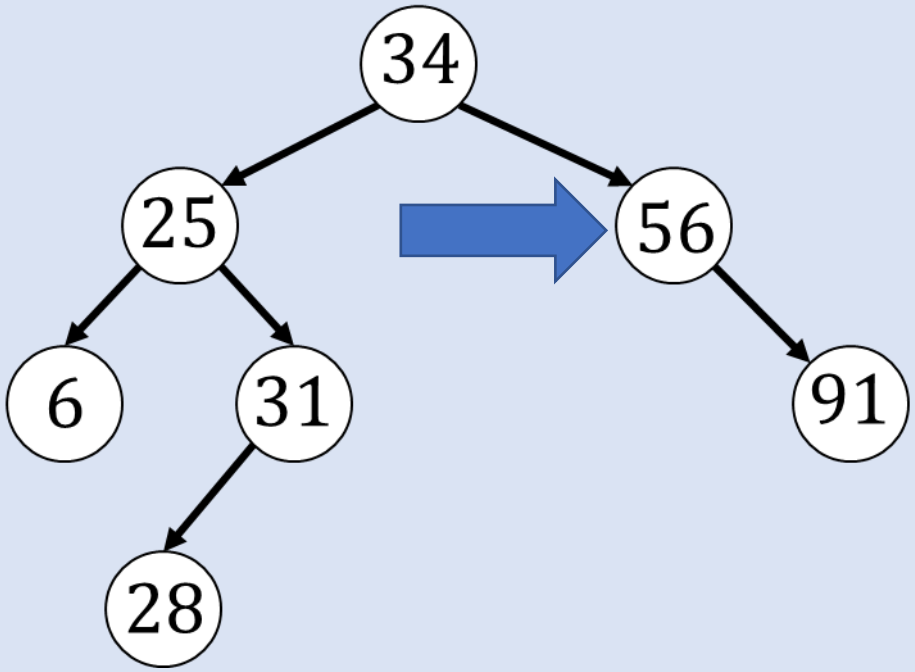
Data Value Processed
6
28
31
25

Postorder Traverse Example



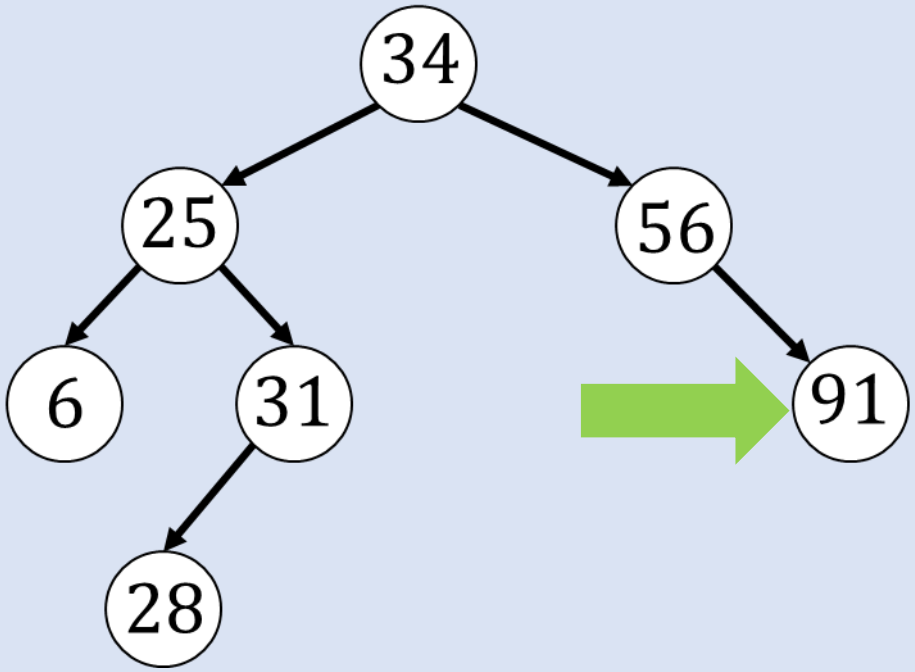
Data Value Processed
6
28
31
25

Postorder Traverse Example



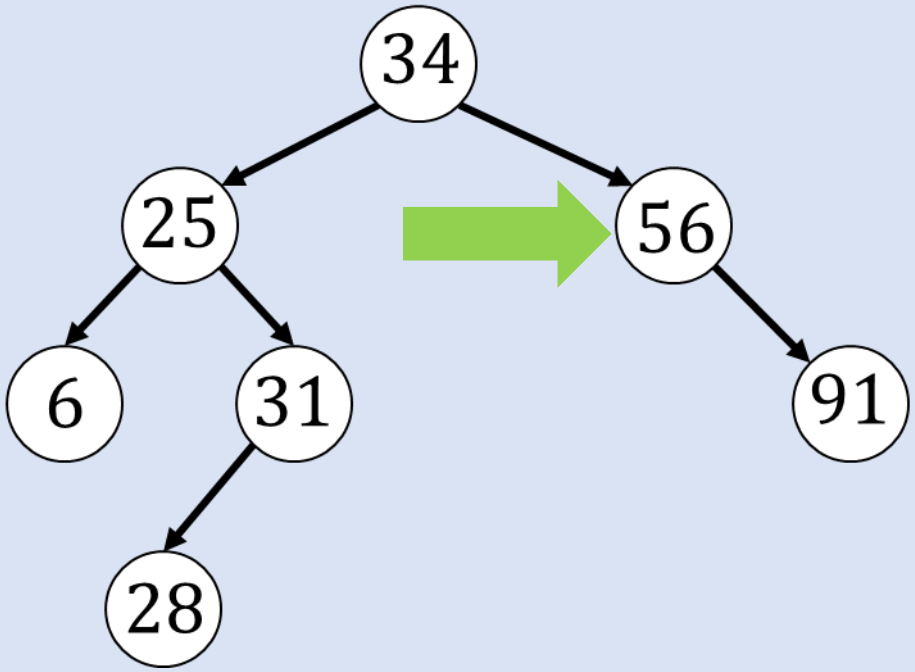
Data Value Processed
6
28
31
25

Postorder Traverse Example



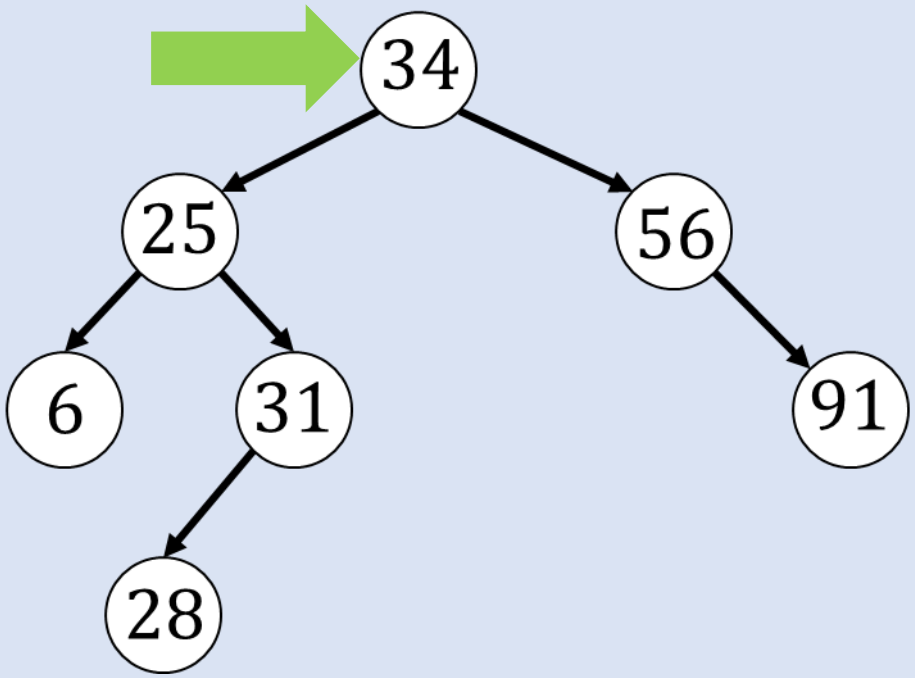
Data Value Processed
6
28
31
25
91

Postorder Traverse Example



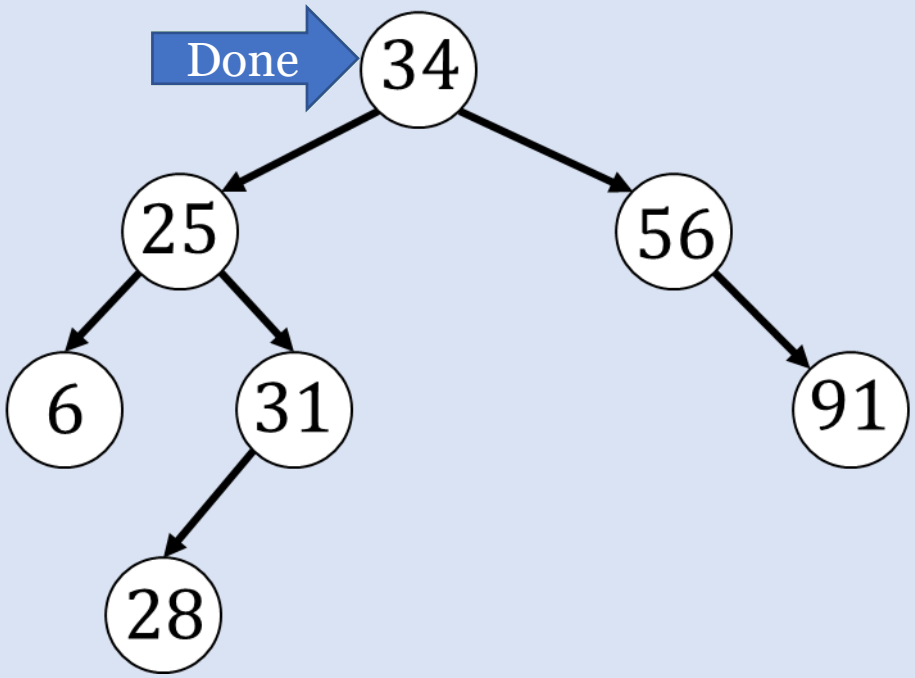
Data Value Processed
6
28
31
25
91
56

Postorder Traverse Example



Data Value Processed
6
28
31
25
91
56
34

Postorder Traverse Example



Data Value Processed
6
28
31
25
91
56
34

Runtime Analysis of BST Operations

- Notes:
  - n represents the number of nodes in the binary search tree
  - h represents the height of the binary search tree
    - $h = \lfloor \log_2 n \rfloor$

Operation	Average Case	Worst Case
Insertion	$O(h)$	$O(n)$
Deletion	$O(h)$	$O(n)$
Search	$O(h)$	$O(n)$
Inorder Traversal	$O(n)$	$O(n)$
Preorder Traversal	$O(n)$	$O(n)$
Postorder Traversal	$O(n)$	$O(n)$